

中国科学技术大学
University of Science and Technology of China

自下而上的解析与翻译

《编译原理和技术(H)》

张昱

yuzhang@ustc.edu.cn

中国科学技术大学

计算机科学与技术学院



中国科学技术大学
University of Science and Technology of China

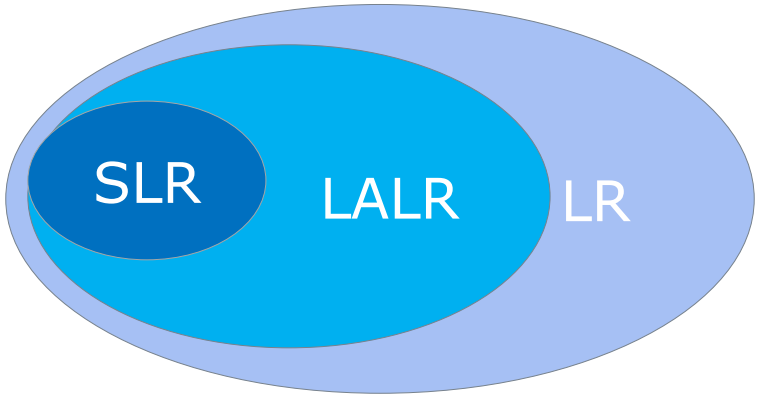
自下而上解析器的生成：LR解析技术

(L-scanning from left to right; R- rightmost derivation in reverse)

LR(k)解析技术

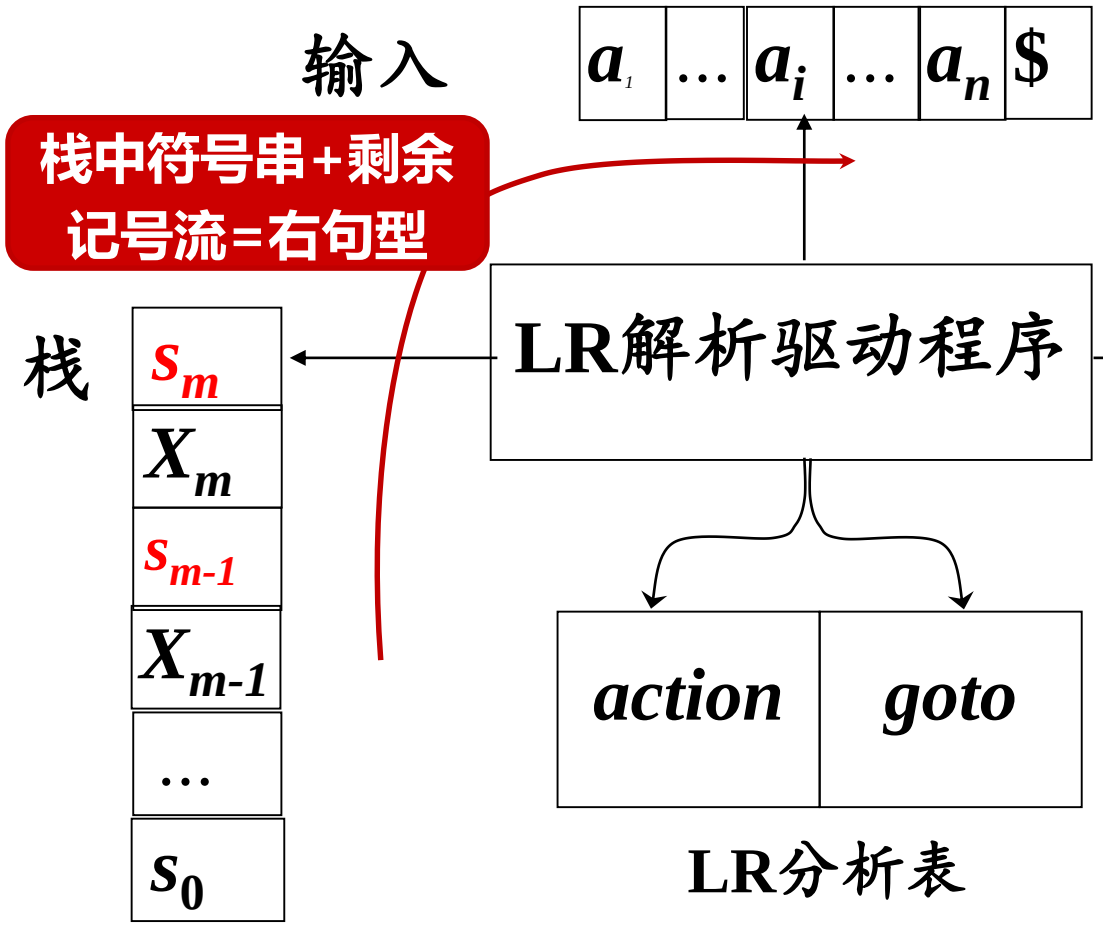
- LR(k)
  - L是指从左向右扫描输入，
  - R是指构造最右推导的逆，
  - *k*是指在决定分析动作时向前查看的符号个数，
    (*k*)省略时，表示*k*是1
- LR解析算法基于LR分析表，后者有三种构造技术
  - SLR：简单的LR方法
  - LALR：向前搜索的LR方法
  - LR：规范的LR方法

三种方法能表达的文法范围如右图所示



LR解析算法: LR系列解析器的模型

- 如何快速识别栈的顶部是否形成句柄？➔引入抽象的**状态**



用**状态**刻画栈中的内容
通过查看栈顶状态，
即可判断是否形成句柄

LR解析器的格局（二元组）

$(s_0X_1s_1X_2s_2\ldots X_ms_m, a_ia_{i+1}\ldots a_n\$)$

(*栈中内容*, *剩余输入*)

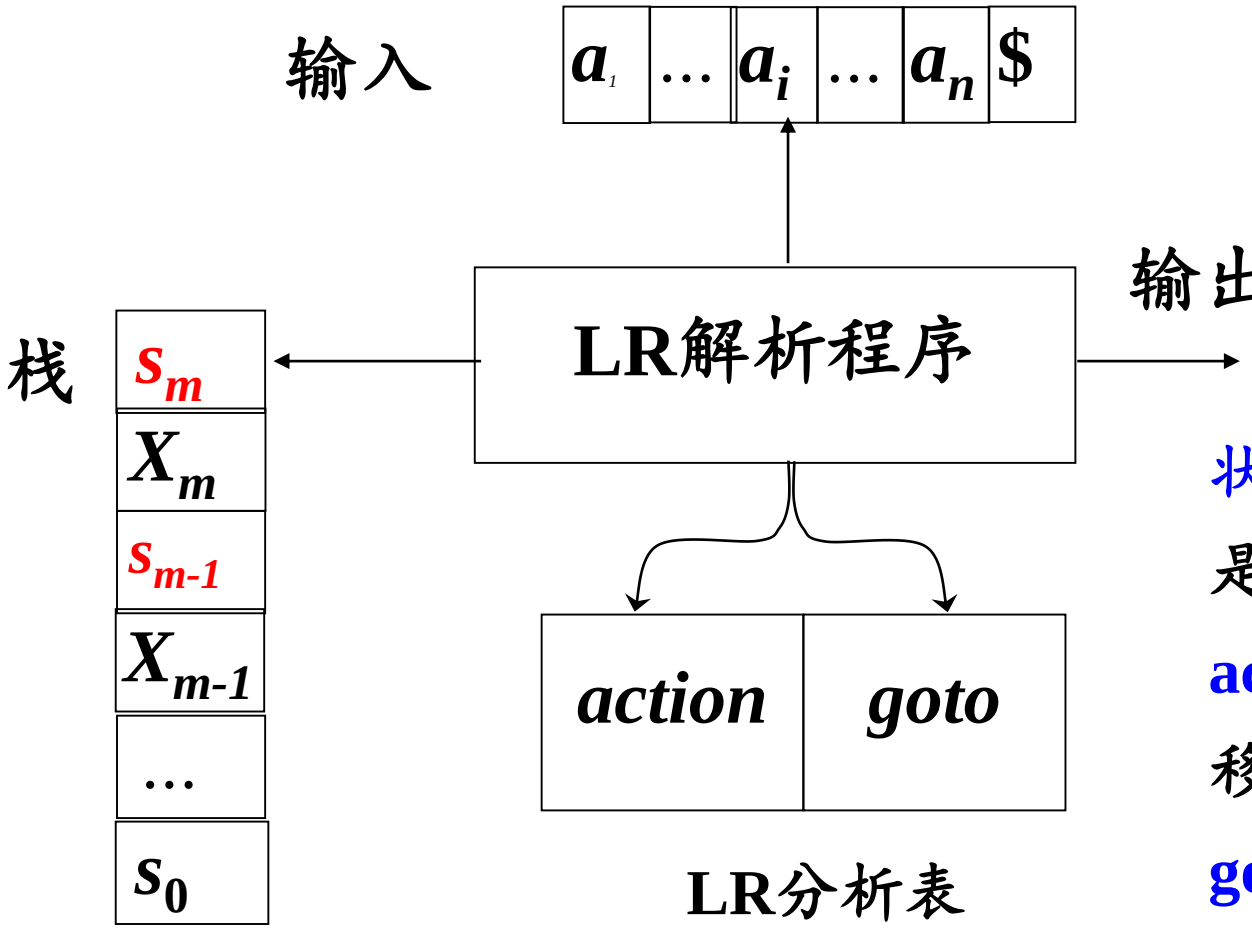
移进-归约解析器

移进符号

LR解析器

移进**状态**和其相关的文法符号(s_0除外)

LR解析算法: LR系列解析器的模型



状态s_j:

是栈中该状态到栈底信息的浓缩

action[s_m, a_i]:

移进|归约|接受|出错

goto[s_{m-r}, A]=s_j:

移进A和s_j (归约后使用)

LR解析: 基本概念

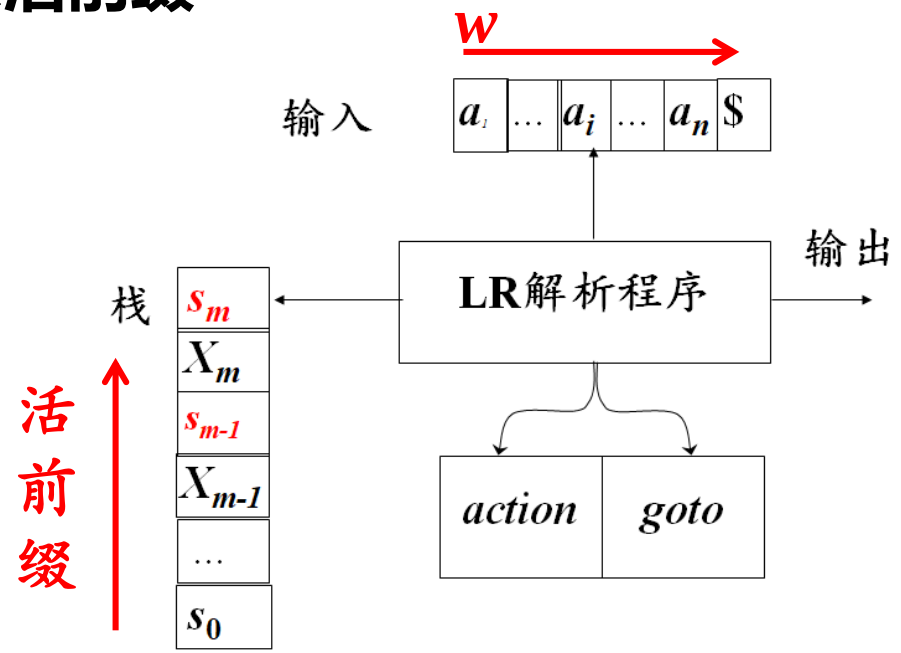
活前缀（viable prefix）

- 右句型的前缀$\gamma\beta$，该前缀不超过最右句柄的右端

$$S \Rightarrow^*_{rm} \gamma A w \Rightarrow_{rm} \gamma \beta w$$

- $\gamma\beta$的任何前缀（包括ε和$\gamma\beta$本身）都是**活前缀**
- w仅包含终结符

- 对应到LR解析模型上的特点
  - **活前缀**：是LR解析栈中从栈底到栈顶的文法符号连接形成的串
  - w：输入缓冲区中剩余的记号串



LR解析算法：举例

实际可不用移进

例题8-1 3.24

$E \to E + T \mid E \to T$

$T \to T * F \mid T \to E$

$F \to (E) \mid F \to \mathrm{id}$

si 移进当前输入符号和状态*i*

rj 按第 *j* 个产生式进行归约

acc 接受

LR分析表

状态	动作						转移		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

LR解析算法：举例

栈	输　入	动　作
0	id * id + id $	

状态	动作						转移		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

LR解析算法：举例

栈	输　入	动　作
0	id * id + id $	移进 (查动作表)
0 id 5	* id + id $	

状态	动作						转移		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

LR解析算法：举例

栈	输　入	动　作
0	id * id + id $	移进
0 id 5	* id + id $	按$F \rightarrow$ id归约
0 F 3	* id + id $	

1. 查action[5, *]= r6 归约
2. 按r6执行归约($F \rightarrow$id)：
   - 从栈中弹出| id |个状态-符号对
   - 查goto[0, F] =>3
   - 将(F, 3)入栈

状态	动作						转移		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

LR解析算法：举例

栈	输　入	动　作
0	id * id + id $	移进
0 id 5	* id + id $	按$F \rightarrow$ id归约
0 F 3	* id + id $	按$T \rightarrow F$归约
0 T 2	* id + id $	

状态	动作						转移		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

LR解析算法：举例

栈	输　入	动　作
0	id * id + id $	移进
0 id 5	* id + id $	按$F \rightarrow$ id归约
0 F 3	* id + id $	按$T \rightarrow F$归约
0 T 2	* id + id $	移进
0 T 2 * 7	id + id $	

状态	动作						转移		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

LR解析算法：举例

状态	动作						转移		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

栈	输　入	
0	id * id + id	
0 id 5	* id + id	
0 *F* 3	* id + id	
0 *T* 2	* id + id $	移进
0 *T* 2 * 7	id + id $	移进
0 *T* 2 * 7 id 5	+ id $	

LR解析算法：举例

状态	动作						转移		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

栈	输　入	
0	id * id + id	
0 id 5	* id + id	
0 *F* 3	* id + id	
0 *T* 2	* id + id $	移进
0 *T* 2 * 7	id + id $	移进
0 *T* 2 * 7 id 5	+ id $	按*F* → id归约
0 *T* 2 * 7 *F* 10	+ id $	

LR解析算法：举例

状态	动作						转移		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

栈	输　入	
0	id * id + id	
0 id 5	* id + id	
0 *F* 3	* id + id	
0 *T* 2	* id + id $	移进
0 *T* 2 * 7	id + id $	移进
0 *T* 2 * 7 id 5	+ id $	按*F* → id归约
0 *T* 2 * 7 *F* 10	+ id $	按*T* → *T***F*归约
. . .	. . .	. . .

LR解析算法：举例

状态	动作						转移		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

栈	输　入	
0	id * id + id	
0 id 5	* id + id	
0 *F* 3	* id + id	
0 *T* 2	* id + id $	移进
0 *T* 2 * 7	id + id $	移进
0 *T* 2 * 7 id 5	+ id $	按$F \rightarrow$ id归约
0 *T* 2 * 7 *F* 10	+ id $	按$T \rightarrow T*F$归约
. . .	. . .	. . .
0 *E* 1	$	接受

归约为 **开始符号**

完成合法输入串的分析

LR解析: 基本概念

- LR文法(LR grammar)
  - 能为之构造出所有条目（若存在）都唯一的LR分析表
- LR分析表
  - 移进+ goto（转移函数）：本质上是识别活前缀的DFA

状态	动　作						转　移		
	id	+	*	(	)	$	*E*	*T*	*F*
0	*s*5			*s*4			1	2	3
1		*s*6				*acc*			
2		*r*2	*s*7		*r*2	*r*2			
3		*r*4	*r*4		*r*4	*r*4			
4	*s*5			*s*4			8	2	3

LR方法与LL方法的比较

	LR(1)方法	LL(1)方法
建立解析树的方式	自下而上	自上而下
归约还是推导	规范归约	最左推导
决定使用产生式的时机		

$$S \Rightarrow \ldots \Rightarrow \delta A\, b\, w \Rightarrow \delta\, l\, \beta\, b\, w$$

$A \rightarrow l\beta$

LL(1)决定用该产生式的位置

LR方法与LL方法的比较

	LR(1)方法	LL(1)方法
建立解析树的方式	自下而上	自上而下
归约还是推导	规范归约	最左推导
决定使用产生式的时机		

$$S \Rightarrow \ldots \Rightarrow \delta A\, b\, w \Rightarrow \delta\, l\, \beta\, b\, w$$

$A \rightarrow l\beta$

LL(1)决定用该产生式的位置

LR(1)决定用该产生式的位置

LR方法与LL方法的比较

	LR(1)方法	LL(1)方法
建立解析树的方式	自下而上	自上而下
归约还是推导	规范归约	最左推导
决定使用产生式的时机	看见产生式右部推出的整个终结符串后，才确定用哪个产生式归约	看见产生式右部推出的第一个终结符后，便要确定用哪个产生式推导

$$S \Rightarrow \ldots \Rightarrow \delta A\, b\, w \Rightarrow \delta\, l\, \beta\, b\, w$$

$A \rightarrow l\beta$

LL(1)决定用该产生式的位置

LR(1)决定用该产生式的位置



中国科学技术大学
University of Science and Technology of China

LR分析表的构造

(L-scanning from left to right; R- rightmost derivation in reverse)

简单的LR(SLR)、规范的LR、向前看LR(LALR)

LR(1)分析表的构造

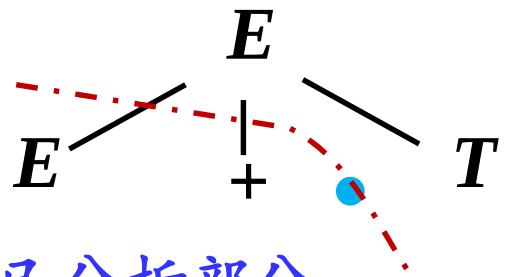
- 文法的LR(0)项目与LR(1)项目
  - 产生式右部加点以刻画分析所处的位置　　$E \rightarrow E + \cdot T$
  - 1：是引入长度为1的搜索符，指示产生式右部之后的符号　$[E \rightarrow E + \cdot T, \$ / +]$
- SLR方法：简单的LR
  - 构造LR(0) 项目集规范族➔形成DFA状态➔SLR分析表
- LR方法：规范的LR
  - 构造LR(1) 项目集规范族➔形成DFA状态➔LR分析表
- LALR方法：向前搜索的LR
  - 构造LR(1) 项目集规范族➔形成DFA状态➔合并同心项目集➔LALR分析表

LR(0)项目和LR(1)项目

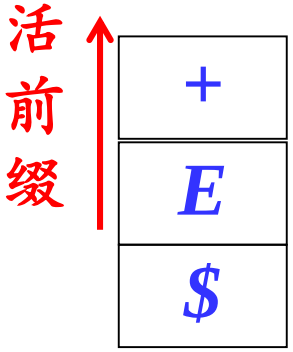
LR(0) 项目

- 在右部的某个地方加点的产生式
- 加点的目的是用来表示分析过程所处的位置 – 刻画分析的状态

$E \rightarrow E + \cdot T$



已分析部分
形成活前缀
在栈中



例 $A \rightarrow XYZ$ 对应有四个LR(0)项目

$A \rightarrow \cdot XYZ$　　$A \rightarrow X \cdot YZ$

$A \rightarrow XY \cdot Z$　　$A \rightarrow XYZ \cdot$

$A \rightarrow \varepsilon$只有一个项目和它对应

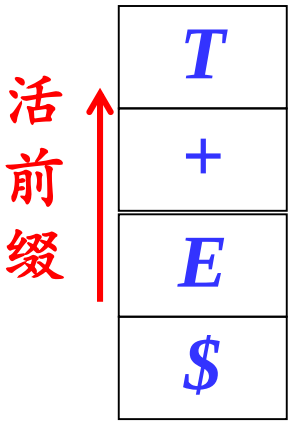
$A \rightarrow \cdot$

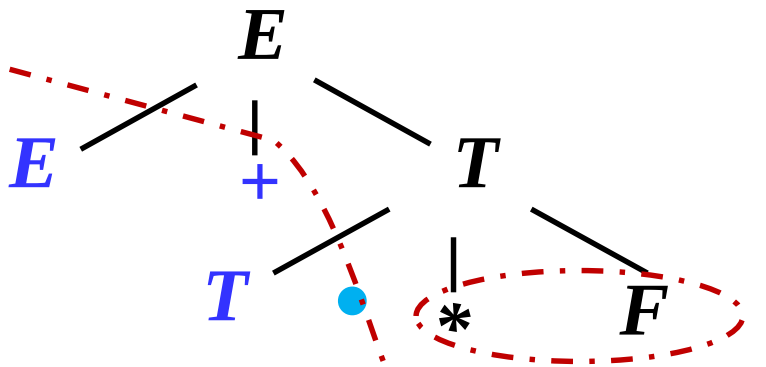
LR(0)项目和LR(1)项目

LR(0) 项目

- 在右部的某个地方加点的产生式
- 加点的目的是用来表示分析过程所处的位置 – 刻画分析的状态

$T \rightarrow T \cdot *F$





尚未分析的部分
对应的终结符串
在输入缓冲区中

LR(0)项目和LR(1)项目

LR(0) 项目

新增产生式和新的开始符号

$S' \to S$
$S \to BB$
$B \to bB \mid a$

满足该语言的句子有：
aa
baa
baba
……

文法拓广：旨在指示解析器

何时开始分析、何时完成分析

$[S' \to \cdot S]$ 初始项目　　$[S' \to S \cdot]$ 结束项目

I_0:

$S' \to \cdot S$
$S \to \cdot BB$
$B \to \cdot bB$
$B \to \cdot a$

核心项目
1）初始项目；
2）点不在最左端的项目

非核心项目
非初始项目且点在最左端的项目

可通过对核心项目求闭包来获得
为节省存储空间，可省去

求LR(0)项目集的闭包closure(*I*) 图3.24

- *I*中的每个项目都加入到closure(*I*)
- $[A \to \alpha \cdot B\beta] \in I$

  $\forall B \to \gamma : [B \to \cdot \gamma] \in \text{closure}(I)$

LR(0)项目和LR(1)项目

LR(0)项目集规范族

$S' \to S$
$S \to BB$
$B \to bB \mid a$

初始项目：$[S' \to \cdot S]$

结束项目：$[S' \to S \cdot]$

I_0:

$S' \to \cdot S$

$S \to \cdot BB$

$B \to \cdot bB$

$B \to \cdot a$

LR(1) 项目：$[A \to \alpha \cdot \beta, a]$

表示A之后紧跟a.

如果$S \Rightarrow^*_{rm} \delta Aw \Rightarrow_{rm} \delta\alpha\beta w$，则$a$是$w$的第一个符号，或者 w 是ε 且a是$

求LR(0)项目集的闭包closure(I)

$[A \to \alpha \cdot B\beta] \in I$

$\forall\ B \to \gamma : [B \to \cdot\ \gamma\ ] \in I$

LR(1)项目集规范族

新增搜索符

初始项目：$[S' \to \cdot S, \$]$

结束项目：$[S' \to S \cdot, \$]$

I_0:

$S' \to \cdot S, \$$

$S \to \cdot BB, \$$

$B \to \cdot bB, a/b$

$B \to \cdot a, a/b$

首终结符

$aa\$$

$baa\$$

$baba\$$

FIRST(B) ={a, b}

求LR(1)项目集的闭包closure(I)

$[A \to \alpha \cdot B\beta, a] \in I$

$\forall\ B \to \gamma : [B \to \cdot\ \gamma, b] \in I, b \in \text{FIRST}(\beta a)$

问题：LR(1)项目数量庞大 =>状态数偏多

LR分析表的构造

1. 拓广文法

$$S' \to S$$
$$S \to BB$$
$$B \to bB \mid a$$

2. 构造LR(0) 项目集规范族或LR(1)项目集规范族

=>构造识别活前缀的DFA

活前缀：某个右句型的一个前缀，该前缀不超过该右句型的最右句柄的右端

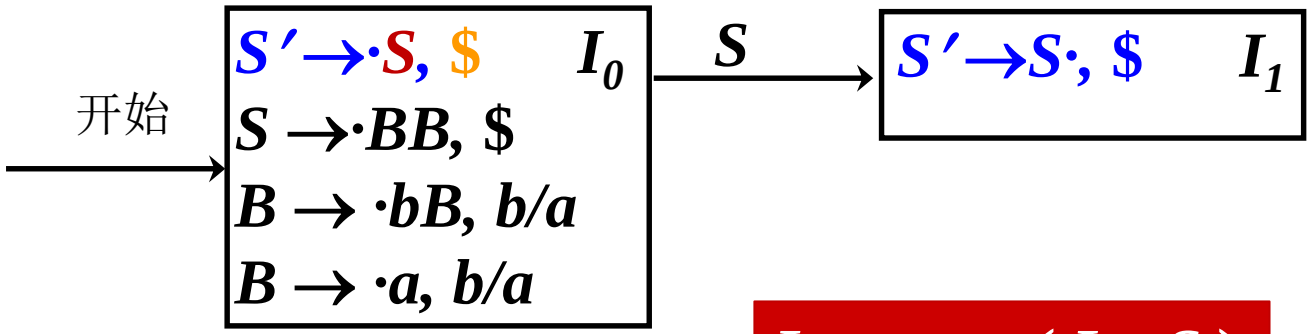
右句型：通过最右推导得到的句型

- **LALR解析（LookAhead LR）**
  - 通过合并同心的LR(1)项目集，得到和SLR同样数量的状态

3. 从上述DFA构造LR分析表

注：
LR(0)项目集规范族 => SLR分析表
LR(1)项目集规范族 => 规范的LR分析表
合并同心的LR(1)项目集规范族 => LALR分析表

构造识别活前缀的DFA (以LR(1)项目集为例)



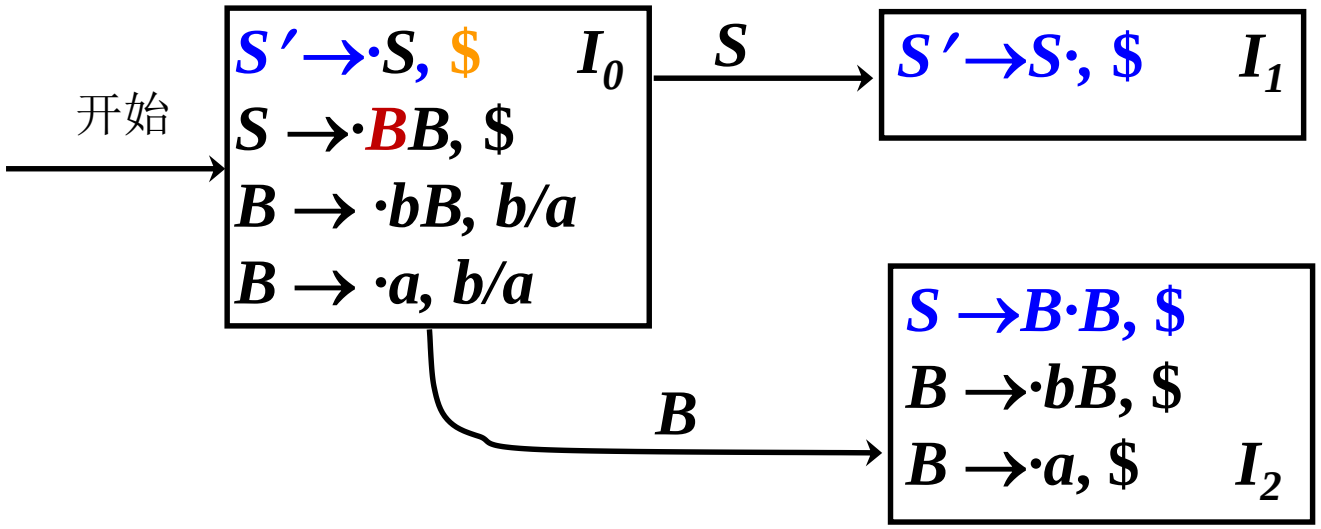
$S' \to S$

$S \to BB$

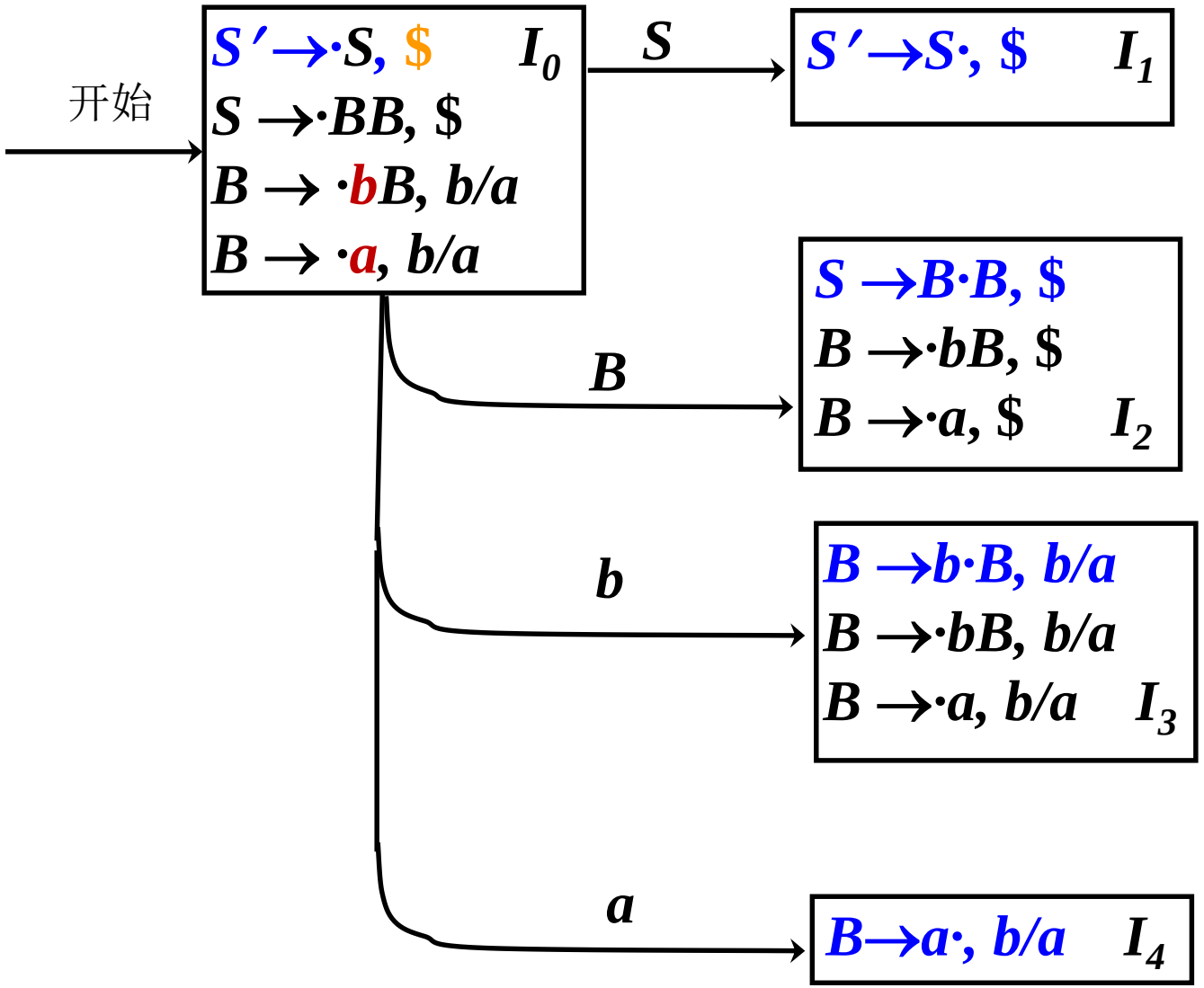
$B \to bB \mid a$

$I_1 := goto\ (\ I_0,\ S\ )$

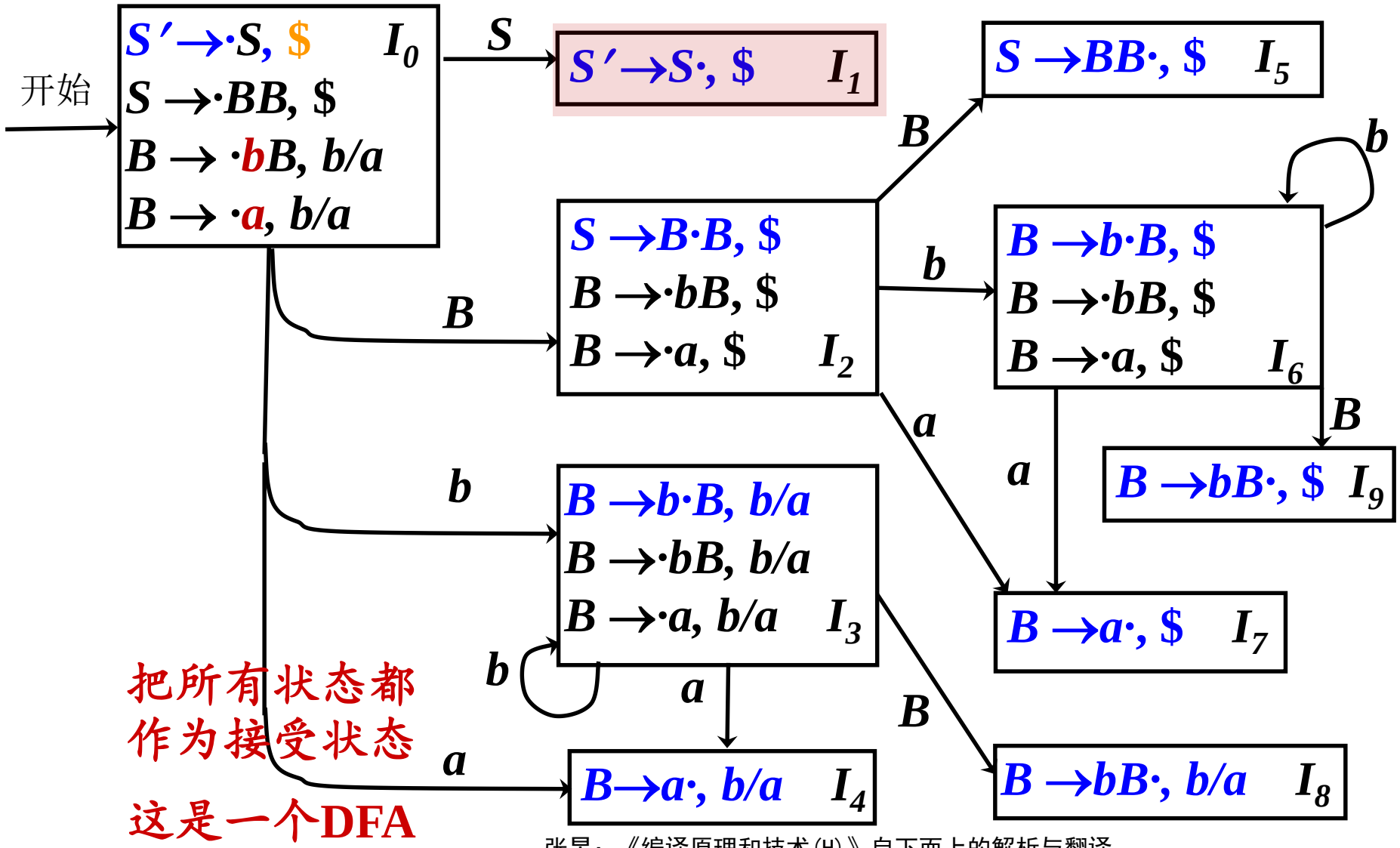
构造识别活前缀的DFA (以LR(1)项目集为例)



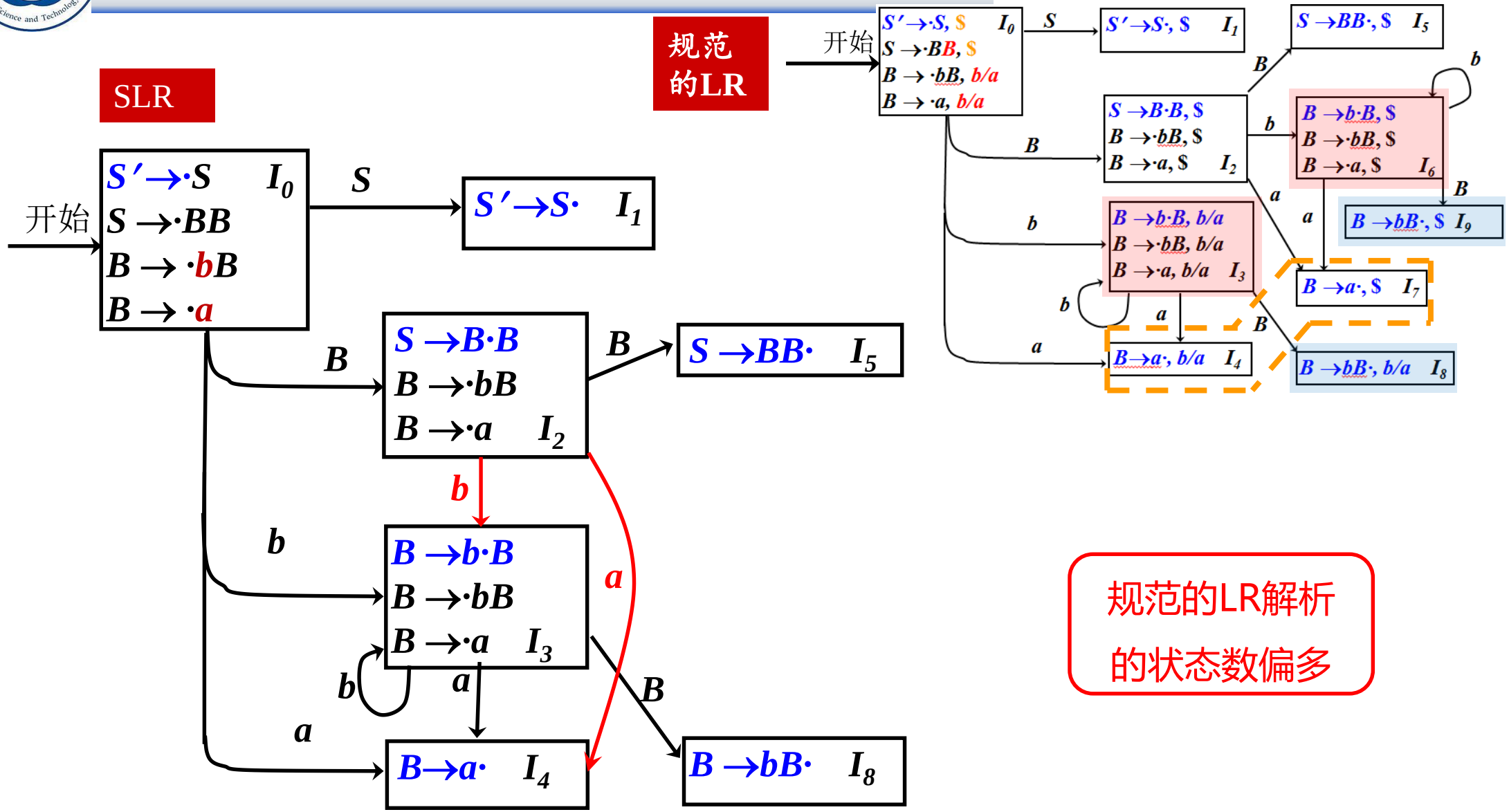
构造识别活前缀的DFA (以LR(1)项目集为例)



构造识别活前缀的DFA (以LR(1)项目集为例)



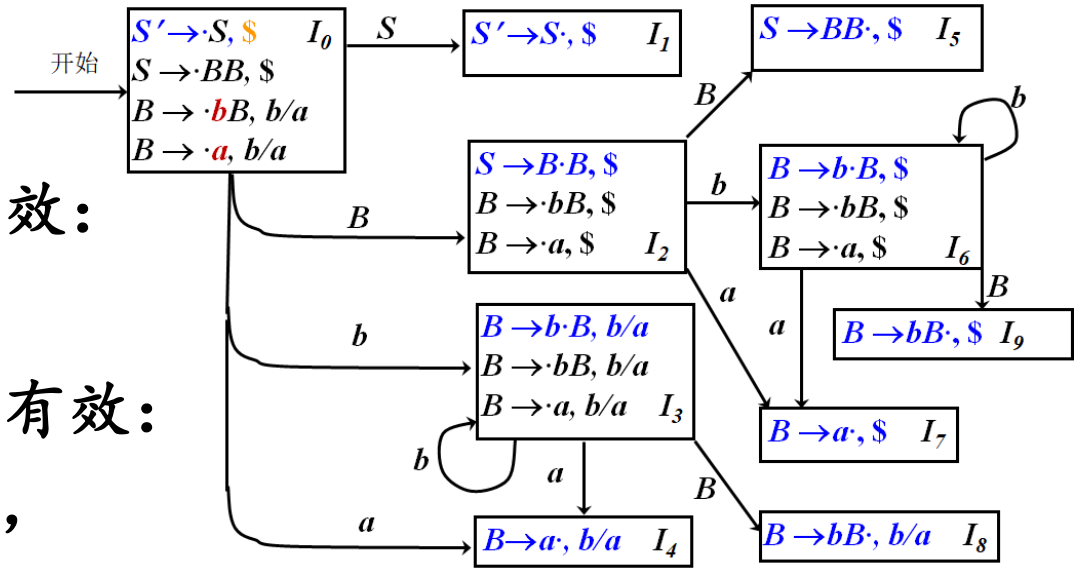
规范的LR vs. SLR 解析



有效项目

项目对活前缀的有效性

- LR(0)项目$[A\to\alpha\cdot\beta]$对活前缀$\gamma=\delta\alpha$有效：
  如果存在推导$S'\Rightarrow^*_{rm}\delta Aw\Rightarrow_{rm}\delta\alpha\beta w$
- LR(1)项目$[A\to\alpha\cdot\beta, a]$对活前缀$\gamma=\delta\alpha$有效：
  如果存在推导$S'\Rightarrow^*_{rm}\delta Aw\Rightarrow_{rm}\delta\alpha\beta w$，
  - a是w的第一个符号，或者w是ε且a是$



项目与活前缀之间的关系

- 活前缀是DFA中从初始状态到项目所在状态路径上的文法符号串联形成的串
- 一个活前缀γ的有效项目集是从这个DFA的初态出发，沿着标记为γ的路径到达的那个项目集（状态）

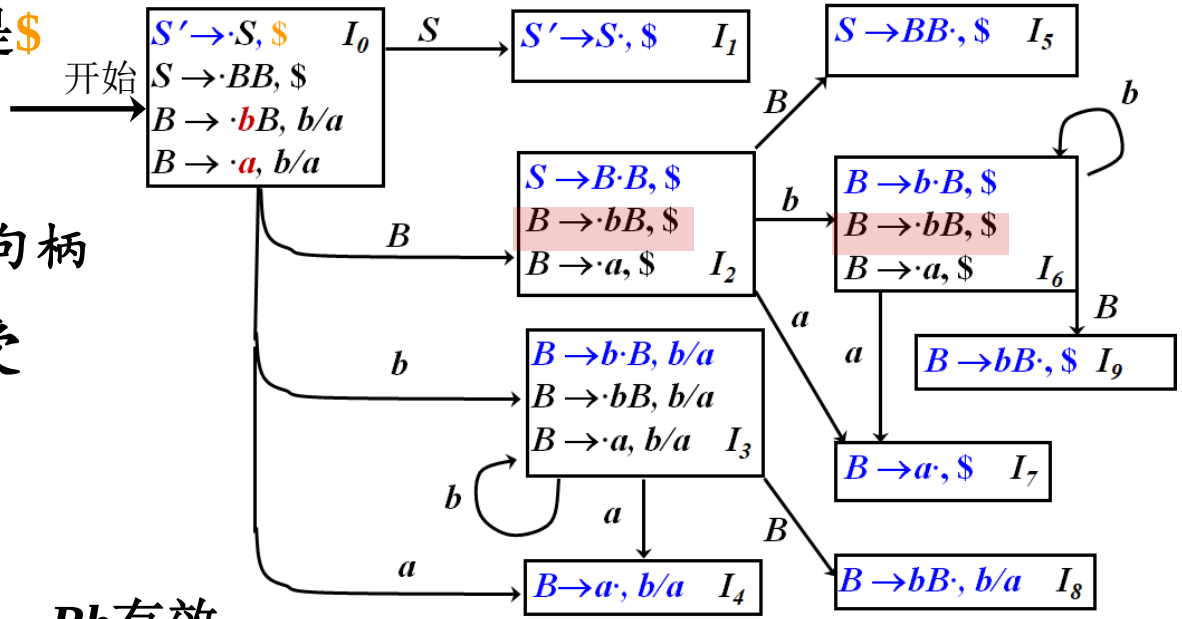
有效项目

- LR(1)项目[$A\to\alpha\cdot\beta, a$]对活前缀 $\gamma=\delta\alpha$ 有效：
  如果存在着推导 $S' \Rightarrow^*_{rm} \delta Aw \Rightarrow_{rm} \delta\alpha\beta w$，其中：
  - a是w的第一个符号，或者 w 是ε 且a是\$

项目与活前缀之间的关系

$S' \Rightarrow S \Rightarrow BB \Rightarrow BbB \Rightarrow BbbB$　　bB是最右句柄

- $BbbB$ 的所有前缀(活前缀)都可接受
  - I_9[$B\to bB\cdot$, \$]对活前缀$BbbB$是有效的
  - I_6[$B\to b\cdot B$, \$]对活前缀Bbb是有效的
  - I_2和I_6中的[$B\to\cdot bB$,\$]分别对活前缀B、Bb有效
- 活前缀B有多个有效项目



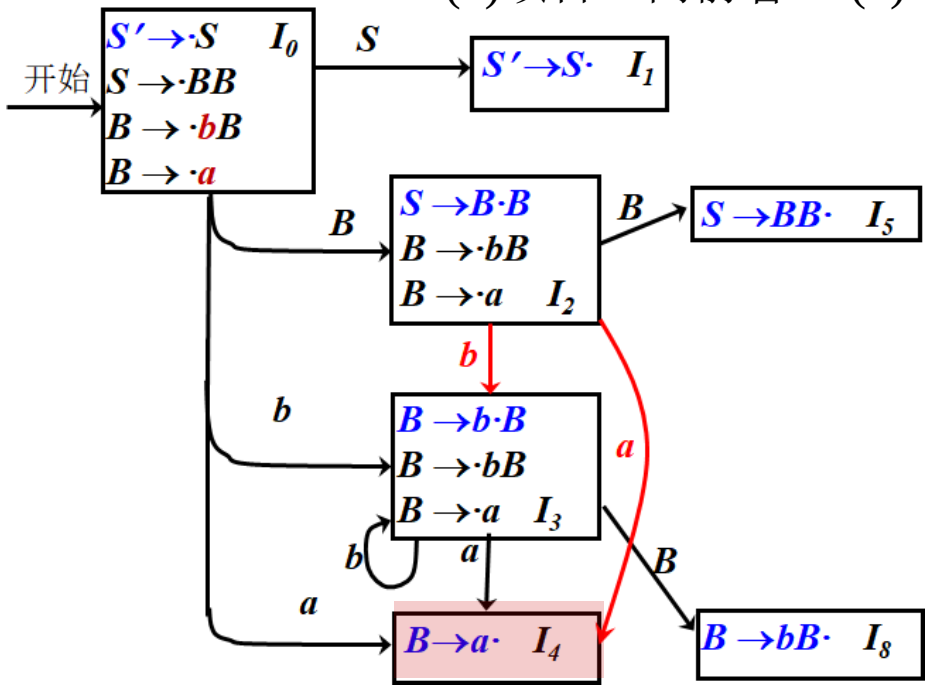
$S \to B\cdot B, \$$
$B \to\cdot bB, \$$
$B \to\cdot a, \$$　I_2

有效项目

- 从LR(0)项目$A \to \alpha \cdot \beta$或LR(1)项目$[A \to \alpha \cdot \beta, a]$对活前缀$\delta\alpha$有效可以知道
  - ✓ 如果$\beta \neq \varepsilon$，应该移进
  - ✓ 如果$\beta = \varepsilon$，应该用产生式$A \to \alpha$归约

  LR(0)项目：向前看Follow(A)中的符号

  LR(1)项目：向前看LR(1) 项目中第2元的符号



Follow(B)={a, b, $}

对于输入a$:

LR(0)项目$B \to a \cdot$：面临a/b/$归约

LR(1)项目$[B \to a \cdot, b/a]$：面临a/b归约

从DFA构造SLR分析表

- 状态 i 从LR(0)项目集 I_i 构造，按如下方法确定*action* 函数：

① 移进：如果$[A\rightarrow\alpha\cdot a\beta]$在 I_i中，并且 $goto(I_i, a) = I_j$，那么置$action[i, a]$为sj

② 归约：如果$[A\rightarrow\alpha\cdot]$在$I_i$中，那么对Follow(A)中所有的$a$, 置$action[i, a]$为$rj$，$j$是产生式 $A\rightarrow\alpha$的编号

③ 接受：如果$[S'\rightarrow S\cdot]$在I_i中，那么置$action[i, \$]$为$acc$

如果出现动作冲突，那么该文法就不是SLR(1)的

- 构造状态i 的$goto$函数
  - 对所有的非终结符A, 如果$goto(I_i, A) = I_j$, 则$goto[i, A] = j$
- 不能由上面两步定义的条目都置为error
- 解析器的初始状态：包含$[S'\rightarrow\cdot S]$的项目集对应的状态

构造规范的LR分析表

- 构造LR分析表，状态 i 的*action*函数按如下确定

① 如果$[A \to \alpha \cdot a\beta, b]$在$I_i$中,且$goto(I_i, a) = I_j$，那么置$action[i, a]$为$sj$

② 如果$[A \to \alpha \cdot, a]$在I_i中，且$A \neq S'$，那么置$action[i, a]$为rj

③ 如果$[S' \to S \cdot, \$]$在$I_i$中，那么置$action[i, \$] = acc$

如果用上面规则构造,出现了冲突，则文法就不是LR(1)的

- *goto*函数的确定：如果$goto(I_i, A) = I_j$, 那么$goto[i, A] = j$
- 用上面规则未能定义的所有条目都置为error
- 初始状态是包含$[S' \to \cdot S, \$]$的项目集对应的状态

SLR是根据Follow(A)来确定归约动作
这里是根据搜索符 (上下文信息)来确定

LALR分析表

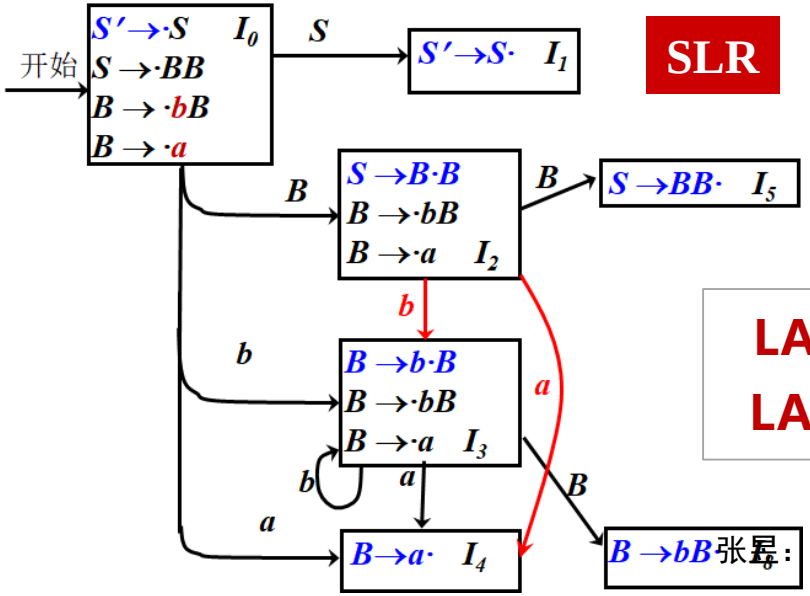
- 同心的LR(1)项目集

  两个项目集在略去搜索符后是相同的集合

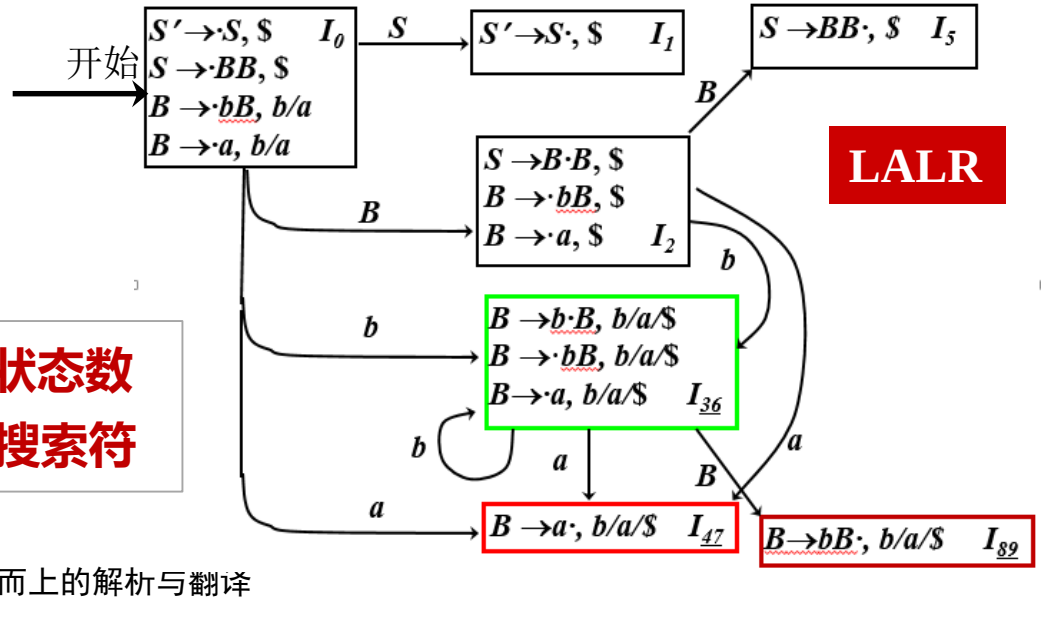
  右图有 3 对同心项目集

  (I3和I6、I4和I7、I8和I9)

- LALR分析表构造方法
  - 通过合并同心的LR(1)项目集来得到



SLR

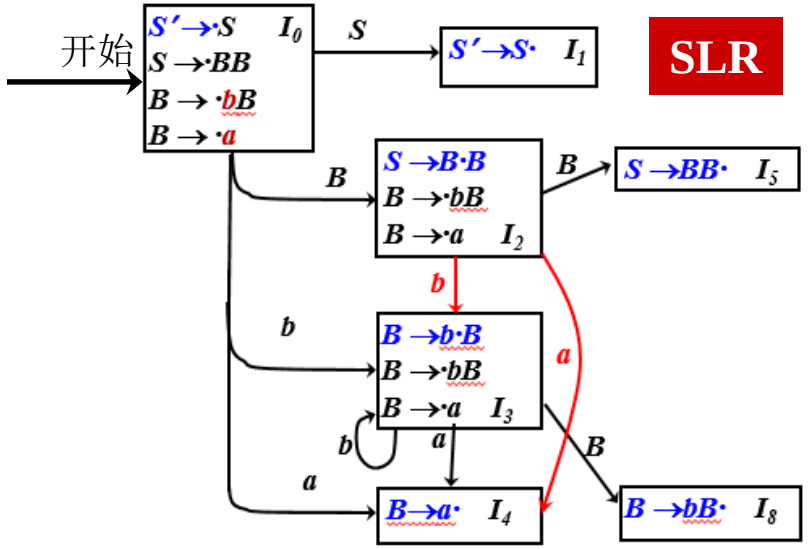


LALR

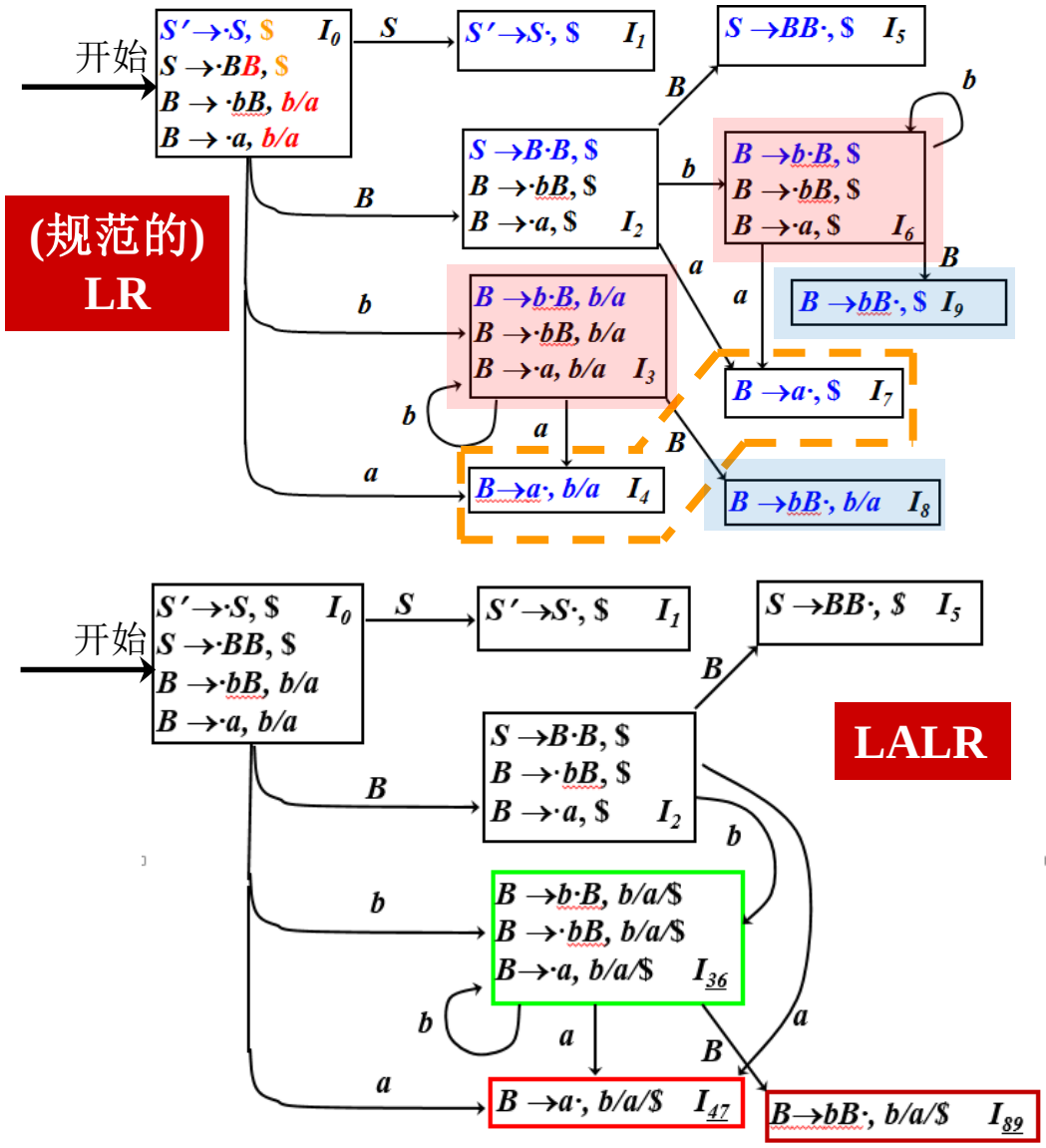
LALR与SLR有相同的状态数

LALR的归约：指定的搜索符

SLR vs. LR vs. LALR解析



SLR



(规范的) LR

LALR

- 同心集的合并不会引起新的移进–归约冲突

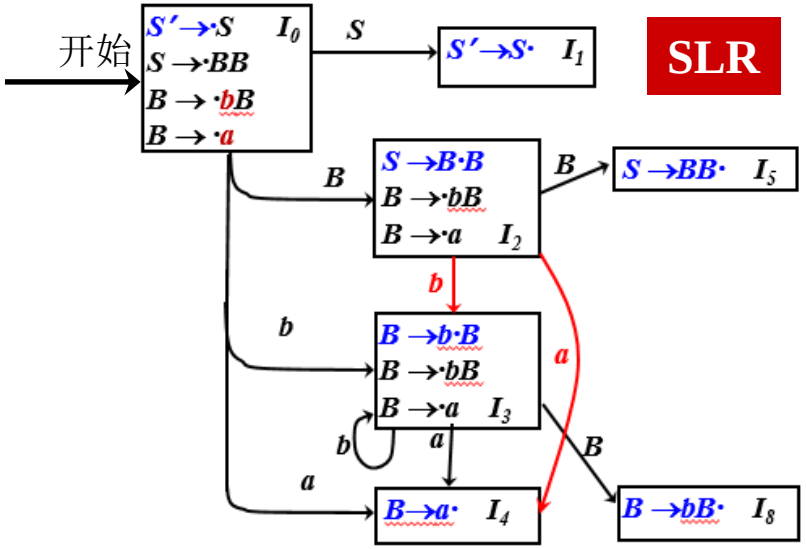
项目集1　　项目集2

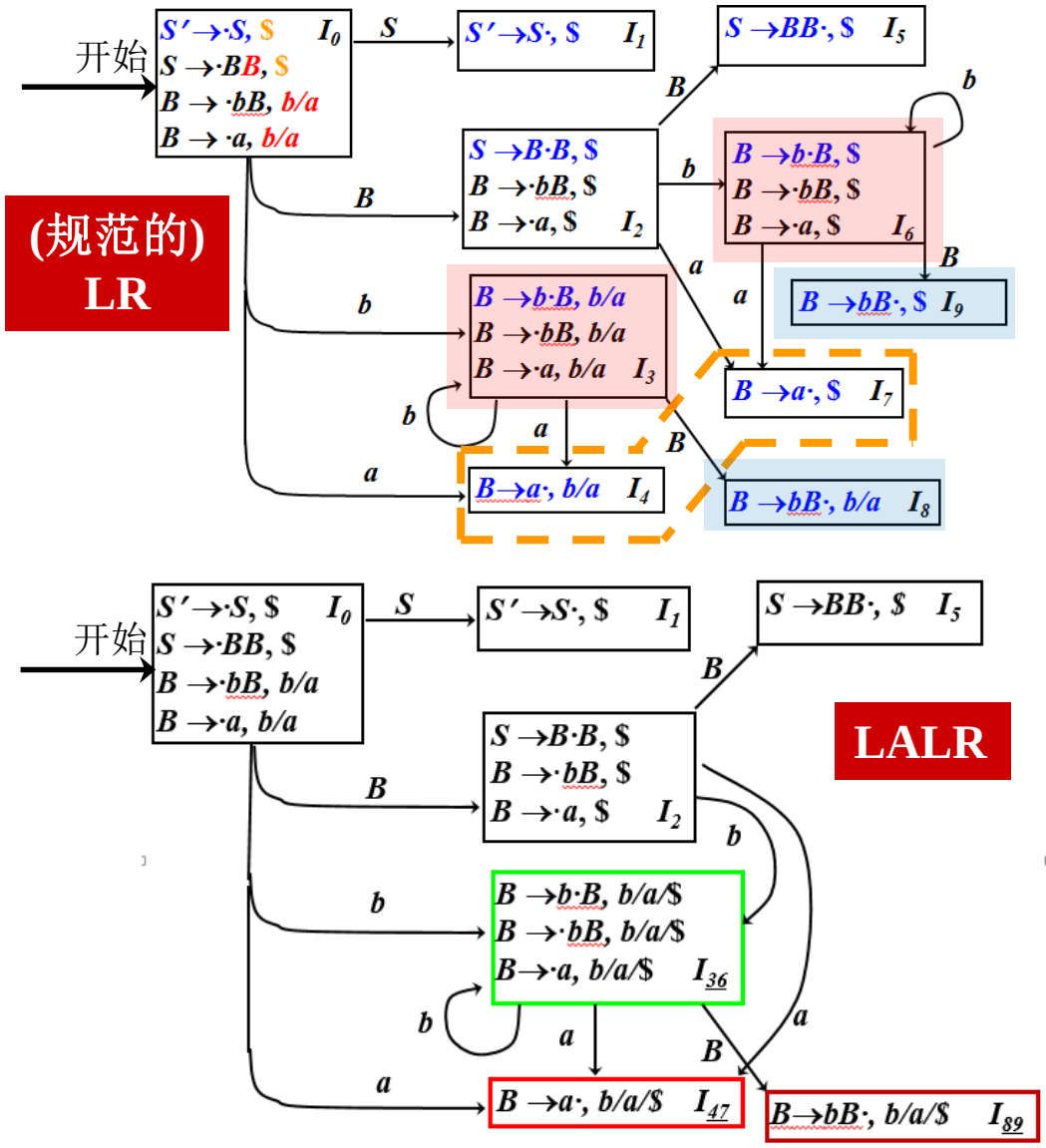
$[A\to\alpha\cdot, a]$　　$[B\to\beta\cdot a\gamma, b]$

…　　…

若合并后有冲突

SLR vs. LR vs. LALR解析





- 同心集的合并不会引起新的移进–归约冲突

项目集1　　项目集2

$[A\to\alpha\cdot, a]$　　$[B\to\beta\cdot a\gamma, b]$

$[B\to\beta\cdot a\gamma, c]$　　$[A\to\alpha\cdot, d]$

…　　…

则合并前就有冲突

LALR vs. LR 解析

- 同心的LR(1)项目集
  - 两个项目集在略去搜索符后是相同的集合
- 同心集的合并不会引起新的移进-归约冲突
- 同心集的合并有可能产生新的归约-归约冲突

$S' \to S$
$S \to aAd \mid bBd \mid aBe \mid bAe$
$A \to c$
$B \to c$

对ac有效的项目集

$$\boxed{\begin{array}{l} A \to c \cdot, d \\ B \to c \cdot, e \end{array}}$$

对bc有效的项目集

$$\boxed{\begin{array}{l} A \to c \cdot, e \\ B \to c \cdot, d \end{array}}$$

合并同心集之后

$$\boxed{\begin{array}{l} A \to c \cdot, d/e \\ B \to c \cdot, d/e \end{array}}$$

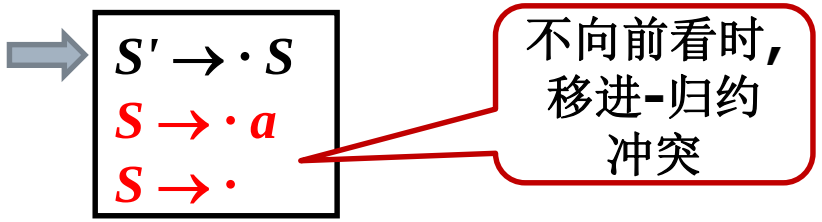
该文法是LR(1)的，
但不是LALR(1)的

LR项目与LR文法小结

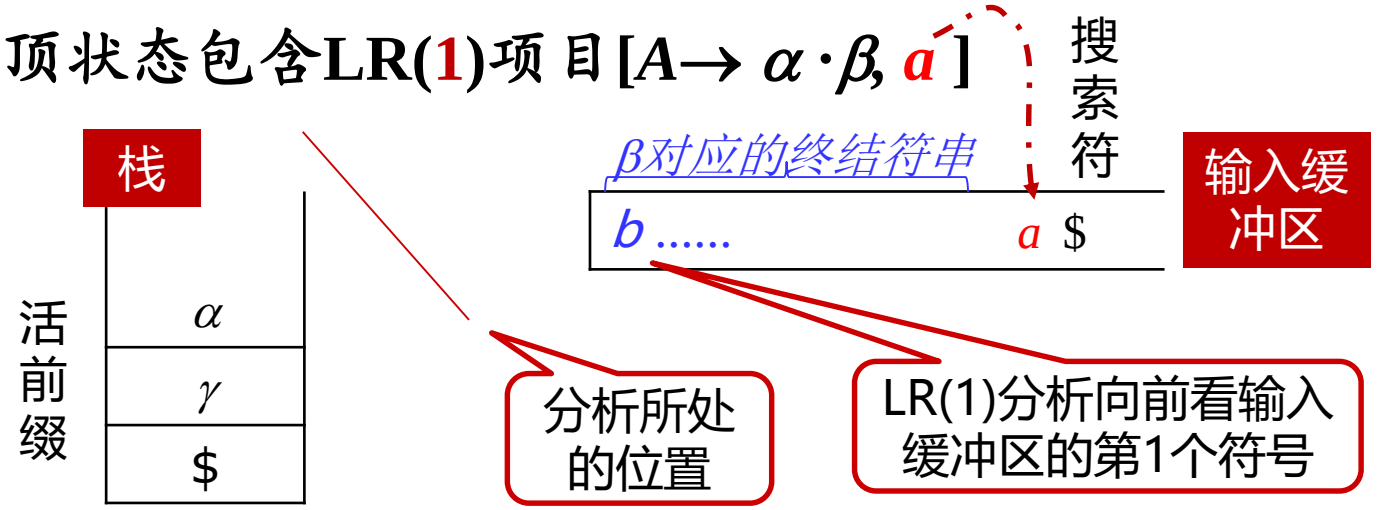
- LR(0)项目[$A\rightarrow \alpha\cdot\beta$]、LR(1)项目[$A\rightarrow \alpha\cdot\beta, a$]
  - 数字表示向前搜索的符号个数，0表示不向前搜索符号
- SLR(k)解析技术与SLR(k)文法
  - SLR(1)解析的状态：LR(0)项目集
  - k是指向前看输入缓冲区的k个符号
- [规范的]LR(k)解析技术与LR(k)文法
  - LR(1)解析的状态：LR(1)项目集
- LALR(k)解析技术与LALR(k)文法
  - LR(1)解析的状态：LR(1)项目集+同心项目集合并

LR项目与LR文法小结

- 不是SLR(0)文法，但是SLR(1)文法
  - 例：$S \to a \mid \varepsilon$



- SLR(0)文法
  - $S \to a$　　$S \to a \mid b$
- 理解LR(1)项目与LR(1)文法中的1
  - 若栈顶状态包含LR(1)项目$[A \to \alpha \cdot \beta, a]$





中国科学技术大学
University of Science and Technology of China

不同LR文法的表达能力

(L-scanning from left to right; R- rightmost derivation in reverse)

简单的LR(SLR)、规范的LR、向前看LR(LALR)

SLR(1)文法的描述能力有限

$S \to V = E$

$S \to E$

$V \to * E$

$V \to \text{id}$

$E \to V$

I_0:
$S' \to \cdot S$
$S \to \cdot V = E$
$S \to \cdot E$
$V \to \cdot * E$
$V \to \cdot \text{id}$
$E \to \cdot V$

V

I_2:
$S \to V \cdot = E$
$E \to V \cdot$

$=$

I_6:
$S \to V = \cdot E$
$E \to \cdot V$
$V \to \cdot * E$
$V \to \cdot \text{id}$

该文法并不是二义的

$S\$ \Rightarrow V = E\$ \Rightarrow * E = E\$$

$S\$ \Rightarrow V = E\$$ 无右句型 $E = E$ ☹

$S\$ \Rightarrow E\$ \Rightarrow V\$$

I_2:

项目$S \to V \cdot = E$使得

action[2,=]=s6

项目$E \to V \cdot$使得

action[2,=]为按$E \to V$归约，

因为Follow(E) = {=, \$}

产生移进-归约冲突

不是SLR(1)但是LR(1)的文法

$S \to V = E$

$S \to E$

$V \to * E$

$V \to \mathrm{id}$

$E \to V$

I_0:

$S' \to \cdot S,\quad \$$

$S \to \cdot V = E,\ \$$

$S \to \cdot E,\quad \$$

$V \to \cdot * E,\quad =$

$V \to \cdot \mathrm{id},\quad =$

$E \to \cdot V,\quad \$$

V →

I_2:

$S \to V \cdot = E,\ \$$

$E \to V \cdot,\quad \$$

$S \to V = E$
$S \to E$
$V \to * E$
$V \to \mathrm{id}$
$E \to V$

I_0:
$S' \to \cdot S$
$S \to \cdot V = E$
$S \to \cdot E$
$V \to \cdot * E$
$V \to \cdot \mathrm{id}$
$E \to \cdot V$

V →

I_2:
$S \to V \cdot = E$
$E \to V \cdot$

$=$ →

I_6:
$S \to V = \cdot E$
$E \to \cdot V$
$V \to \cdot * E$
$V \to \cdot \mathrm{id}$

该文法并不是二义的

$S\$ \Rightarrow V = E\$ \Rightarrow * E = E\$$
$S\$ \Rightarrow V = E\$$ 无句型 $E = E$ ☹
$S\$ \Rightarrow E\$ \Rightarrow V\$$

项目$S \to V \cdot = E$使得
action[2,=]=s6
项目$E \to V \cdot$使得
action[2,=]为按$E \to V$归约，
因为Follow(E)={=, $}
产生移进-归约冲突

LR(1)解析
无移进-归约冲突

非LR的上下文无关结构

若自左向右扫描的移进–归约解析器能及时识别出现在栈顶的句柄，那么相应的文法就是LR（指规范的LR）的。

语言$L = \{ww^R \mid w \in (a \mid b)^*\}$的文法

$S \rightarrow aSa \mid bSb \mid \varepsilon$

不是LR的　　*ababbbbaba*

语言$L = \{wcw^R \mid w \in (a \mid b)^*\}$的文法

$S \rightarrow aSa \mid bSb \mid c$

是LR的　　*ababbcbbaba*

非LR的上下文无关结构

若自左向右扫描的移进–归约解析器能及时识别出现在栈顶的句柄，那么相应的文法就是LR（指规范的LR）的。

语言$L = \{ww^R \mid w \in (a \mid b)^*\}$的文法

$S \rightarrow aSa \mid bSb \mid \varepsilon$

不是LR的　　*ababbbbaba*

I_0：

$S' \rightarrow \cdot S$

$S \rightarrow \cdot aSa$ ⟹ 面临a，*移进*

$S \rightarrow \cdot$ ⟹ $\because a \in Follow(S)$，$\therefore$面临a，*归约*

$S \rightarrow \cdot bSb$

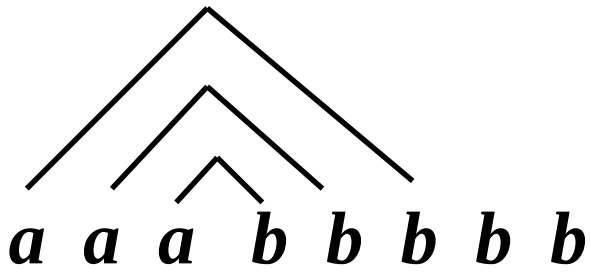
存在移进-归约冲突

故不是SLR(1)文法

例题8-2 写不同的文法

为语言$L=\{a^mb^n \mid n>m\geq 0\}$写三个文法,它们分别是LR(1)的、二义的和非二义且非LR(1)的。

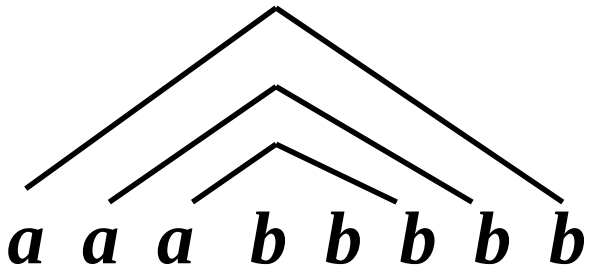
- LR(1)文法：$S \rightarrow AB \quad A \rightarrow aAb \mid \varepsilon \quad B \rightarrow Bb \mid b$
  - A是每个a有个唯一的匹配的b，形成中心对称
  - B是剩余$(n-m)$个b

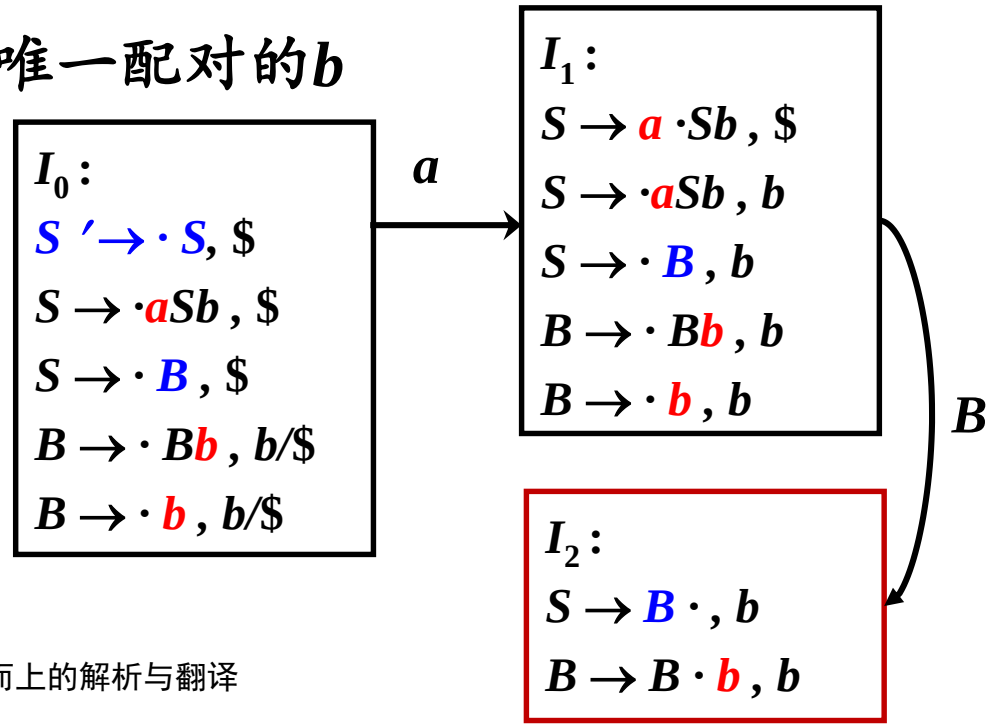


例题8-2 写不同的文法

为语言$L=\{a^m b^n \mid n>m\geq 0\}$写三个文法,它们分别是LR(1)的、二义的和非二义且非LR(1)的。

- LR(1)文法：$S\rightarrow AB \quad A\rightarrow aAb \mid \varepsilon \quad B\rightarrow Bb \mid b$
- 非二义且非LR(1)的文法：$S\rightarrow aSb \mid B \quad B\rightarrow Bb \mid b$
  - 多出来的$(n-m)$个b位于中间，每个a有唯一配对的b
  - 移进-归约冲突：面临第2个b起

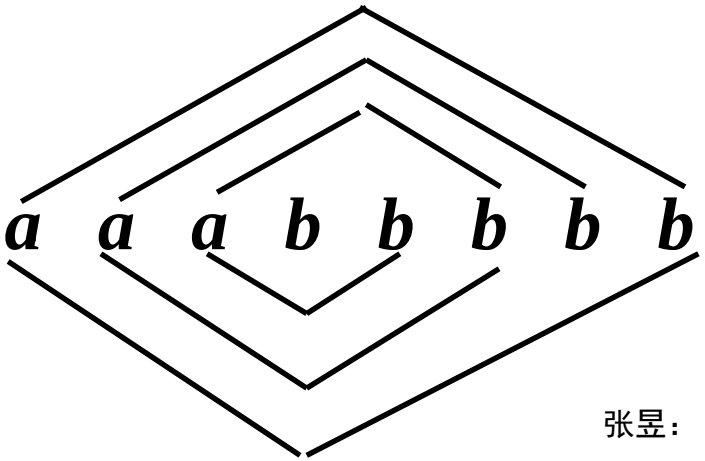


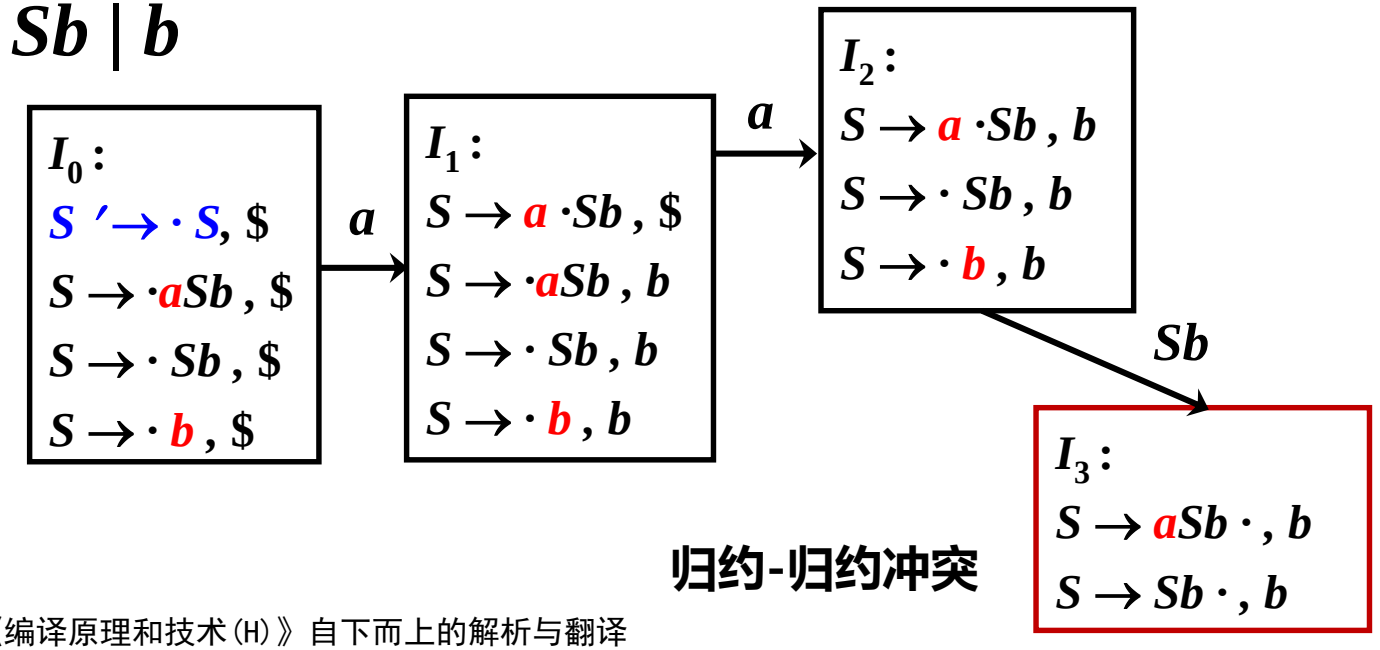


例题8-2 写不同的文法

为语言$L=\{a^m b^n \mid n>m\geq 0\}$写三个文法,它们分别是LR(1)的、二义的和非二义且非LR(1)的。

- LR(1)文法：$S\rightarrow AB \quad A\rightarrow aAb \mid \varepsilon \quad B\rightarrow Bb \mid b$
- 非二义且非LR(1)的文法：$S\rightarrow aSb \mid B \quad B\rightarrow Bb \mid b$
- 二义的文法：$S\rightarrow aSb \mid Sb \mid b$
  - 每个a可有多个配对的b



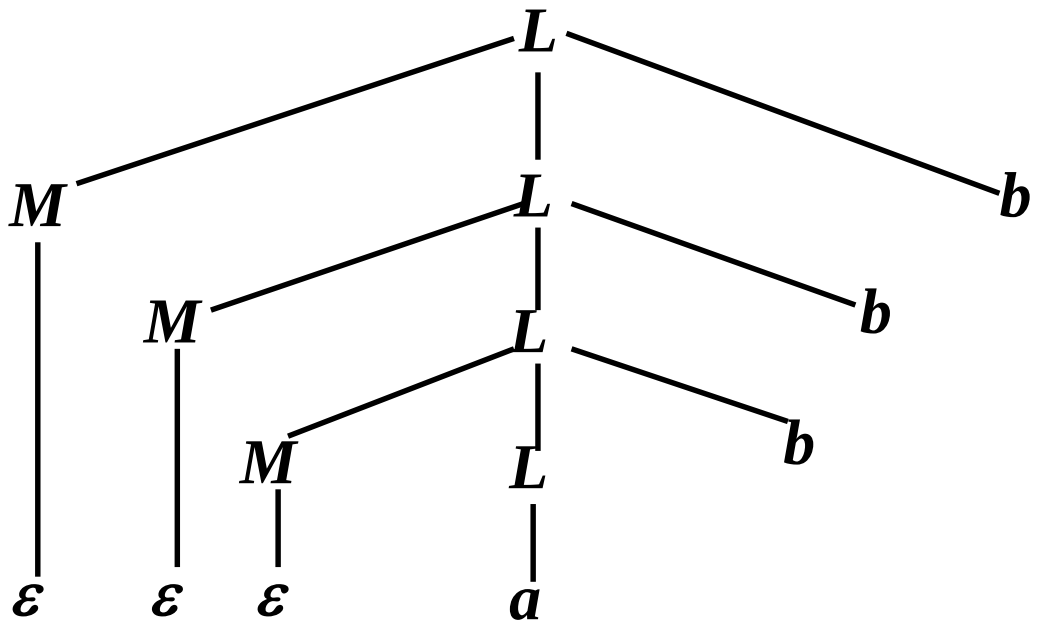


例题8-3

试说明下面文法不是LR(1)的：

$L \rightarrow M\,L\,b \mid a$

$M \rightarrow \varepsilon$



句子*abbb*的解析树

面临a时，不知道该做多少次空归约$M \rightarrow \varepsilon$

例题8-4

下面的文法不是LR(1)的，对它略做修改，使之成为一个等价的SLR(1)文法

program → begin *declist* ; *statement* end

declist → d ; *declist* | d

statement → s ; *statement* | s

该文法产生的句子的形式是

begin d ; d ; … ; d ; s ; s ; … ; s end

修改后的文法如下：

program → begin *declist* *statement* end

declist → d ; *declist* | d ;

statement → s ; *statement* | s

例题8-5

一个C语言的文件如下，第四行的if误写成fi：

```
long gcd(p,q)
long p,q;
{
    fi (p%q == 0)
            return q;
    else
            return gcd(q, p%q);
}
```

基于LALR（1）方法的一个编译器的报错情况如下：

parse error before ‘return’ (line 5).

是否违反了LR解析的活前缀性质？



中国科学技术大学
University of Science and Technology of China

S属性定义的自下而上计算

- 语法树及其构造
  - 语法制导定义 vs. 翻译方案



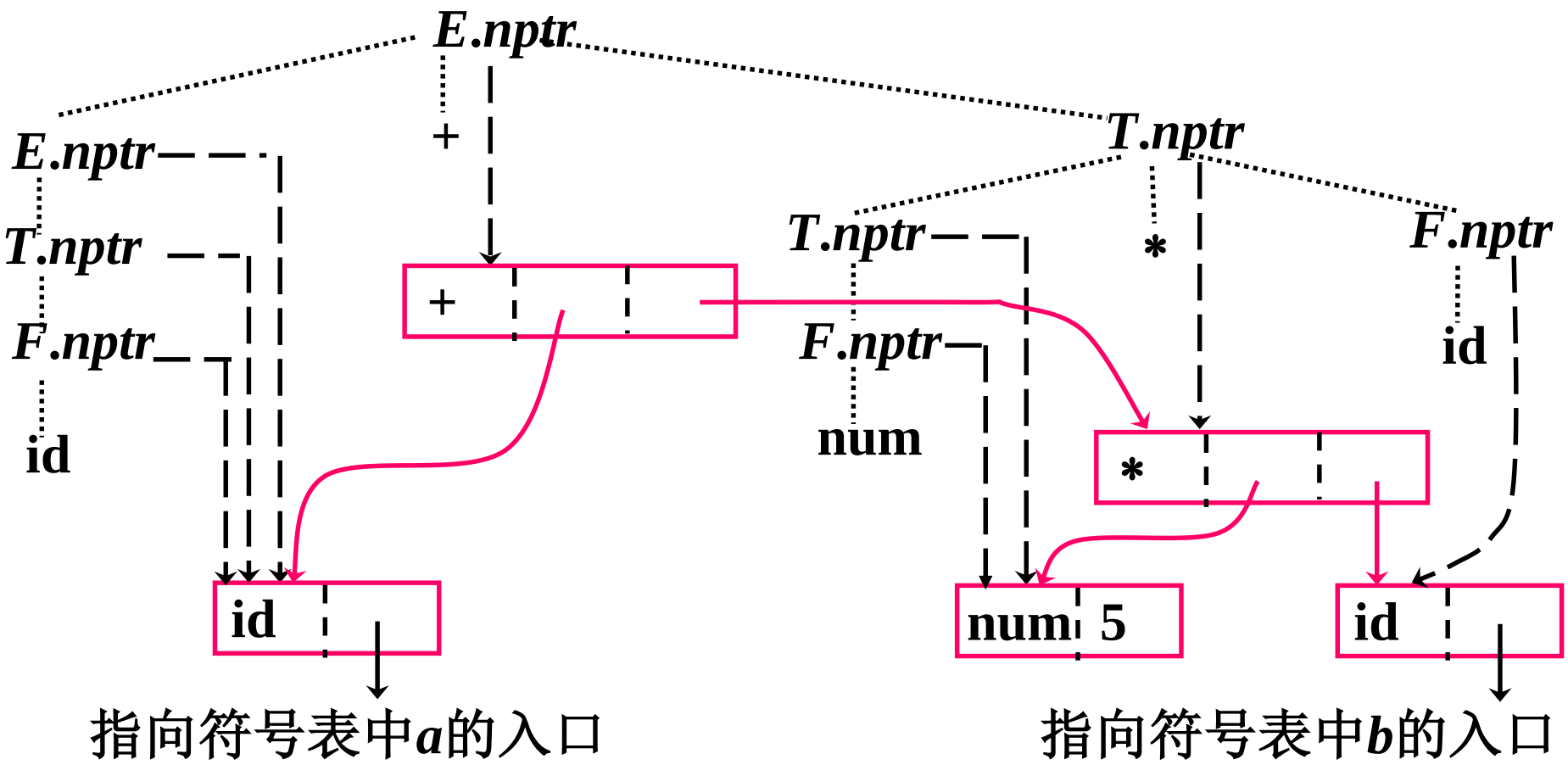
语法制导定义: 构造语法树

产　生　式	语　义　规　则
$E \rightarrow E_1 + T$	$E.nptr = mkNode(\text{'+'}, E_1.nptr, T.nptr)$
$E \rightarrow T$	$E.nptr = T.nptr$
$T \rightarrow T_1 * F$	$T.nptr = mkNode(\text{'*'}, T_1.nptr, F.nptr)$
$T \rightarrow F$	$T.nptr = F.nptr$
$F \rightarrow (E)$	$F.nptr = E.nptr$
$F \rightarrow$ id	$F.nptr = mkLeaf$ (id, id.*entry*)
$F \rightarrow$ num	$F.nptr = mkLeaf$ (num, num.*val*)

参见：bison-examples.tar.gz 中的**config/asgn2ast.y, asgn.lex**

语法树的构造

a+5**b*的语法树的构造



实际执行的函数调用序列：

p_1 = *mkleaf*(**id**, *entrya*);

p_2 = *mkleaf*(**num**, 5);

p_3 = *mkleaf*(**id**, *entryb*);

p_4 = *mknode*('*', p_2, p_3);

p_5 = *mknode*('+', p_1, p_4);

翻译方案

构造语法树的翻译方案（左递归文法）

语义动作，
置于花括号中

$E \rightarrow E_1 + T$	$\{E.nptr = mkNode(\text{‘+’}, E_1.nptr, T.nptr);\}$
$E \rightarrow T$	$\{E.nptr = T.nptr;\}$
$T \rightarrow T_1 * F$	$\{T.nptr = mkNode(\text{‘*’}, T_1.nptr, F.nptr);\}$
$T \rightarrow F$	$\{T.nptr = F.nptr;\}$
$F \rightarrow (E)$	$\{F.nptr = E.nptr;\}$
$F \rightarrow \text{id}$	$\{F.nptr = mkLeaf(\text{id}, \text{id}.entry);\}$
$F \rightarrow \text{num}$	$\{F.nptr = mkLeaf(\text{num}, \text{num}.val);\}$

综合属性的计算规则置于产生式右部的右边，表示识别出右部后计算

S属性的自下而上计算

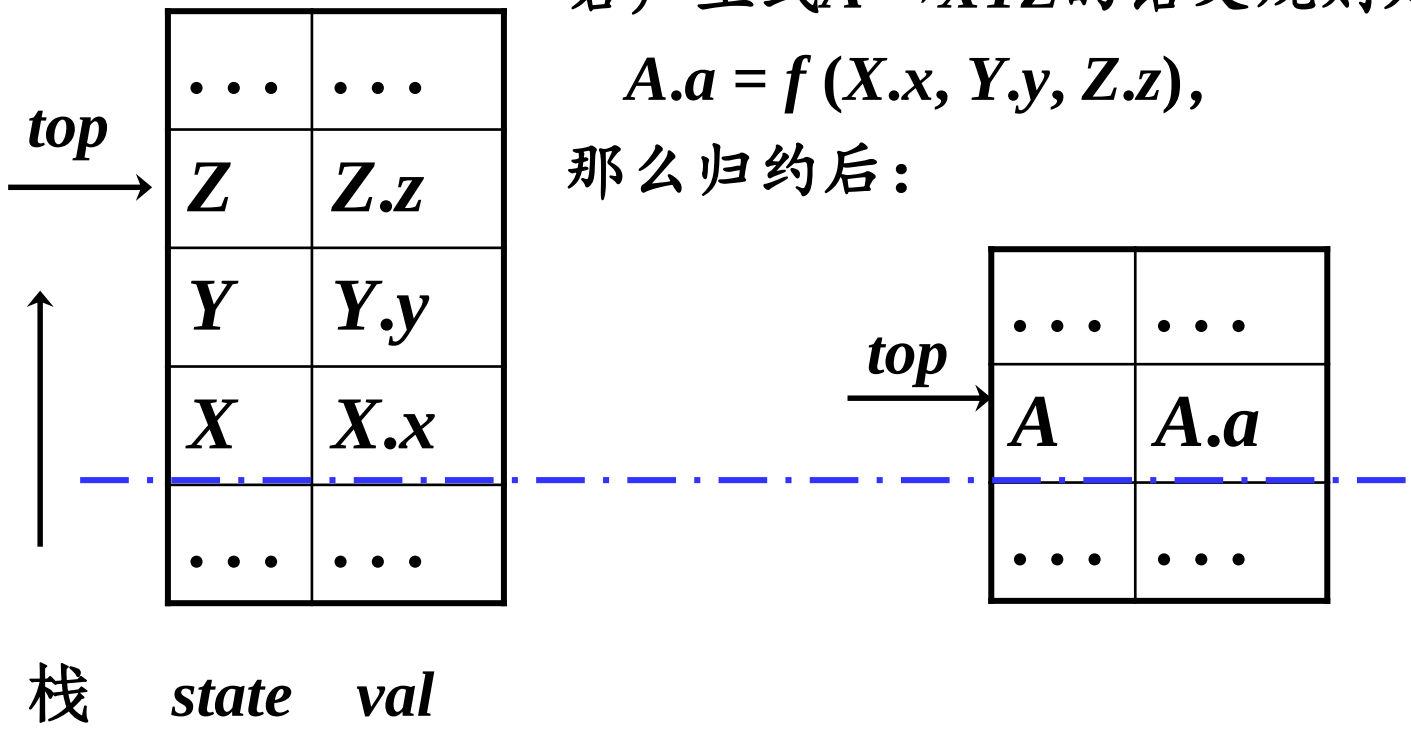
边解析边计算

LR解析器的栈增加一个域来保存综合属性值

若产生式$A \rightarrow XYZ$的语义规则是

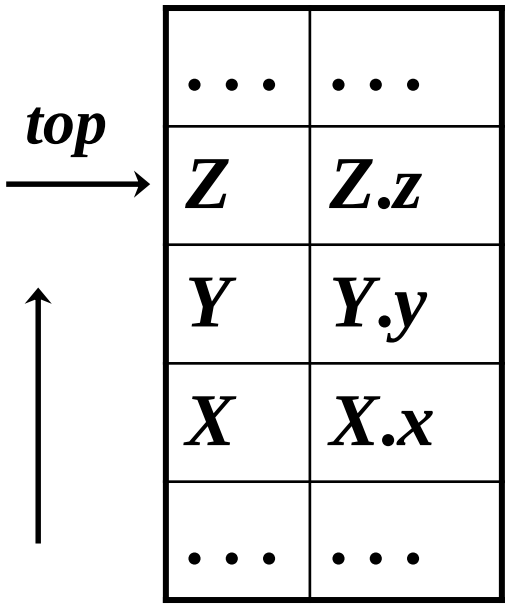
$A.a = f(X.x, Y.y, Z.z)$,

那么归约后：



自下而上的翻译

例　简单计算器的语法制导定义改成栈操作代码

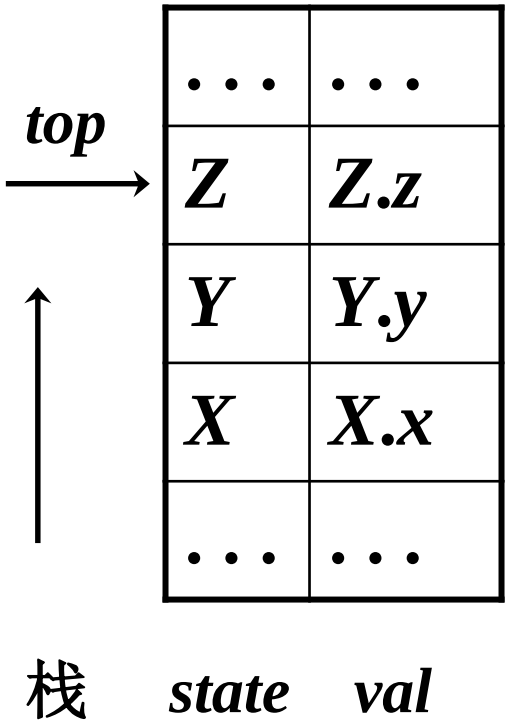


产 生 式	语 义 规 则
$L \rightarrow E$ n	*print* (*E.val*)
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow$ digit	$F.val$ = digit.*lexval*

参见：bison-examples.tar.gz 中的config/expr1.y, expr.lex

自下而上的翻译

例　简单计算器的语法制导定义改成栈操作代码

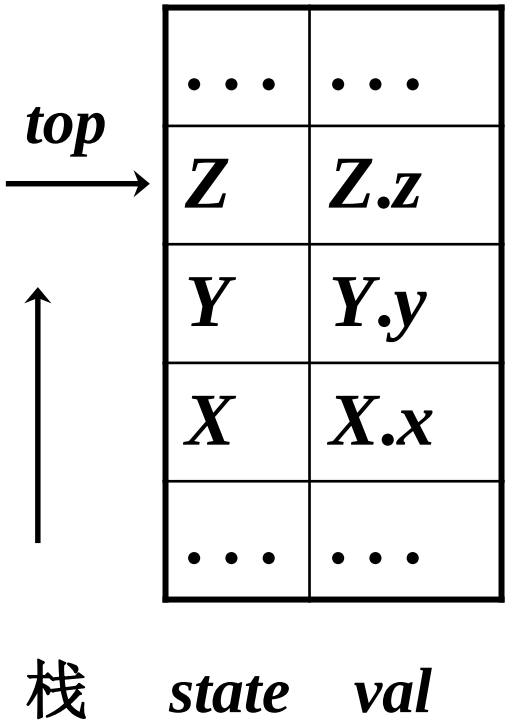


产 生 式	代 码 段
$L \rightarrow E$ n	*print* (*val*[*top*–1])
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow$ digit	$F.val =$ digit.*lexval*

注：栈顶位置指示器top的修改由原来的解析程序在语义动作执行后去做

自下而上的翻译

例 简单计算器的语法制导定义改成栈操作代码

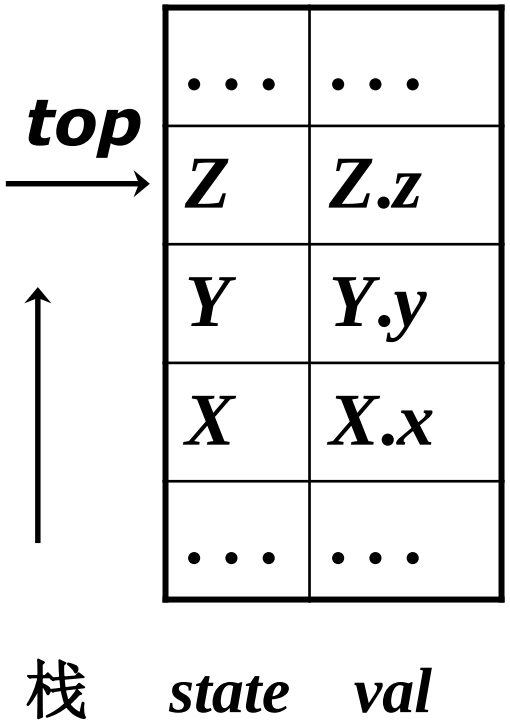


产 生 式	代 码 段
$L \rightarrow E$ n	*print* (*val*[*top*–1]);
$E \rightarrow E_1 + T$	*val*[*top*–2] = *val* [*top*–2] +*val*[*top*];
$E \rightarrow T$	*E.val* = *T.val*
$T \rightarrow T_1 * F$	*T.val* = T_1.*val* * *F.val*
$T \rightarrow F$	*T.val* = *F.val*
$F \rightarrow (E)$	*F.val* = *E.val*
$F \rightarrow$ digit	*F.val* = digit.*lexval*

注：栈顶位置指示器top的修改由原来的解析程序在语义动作执行后去做

自下而上的翻译

例　简单计算器的语法制导定义改成栈操作代码

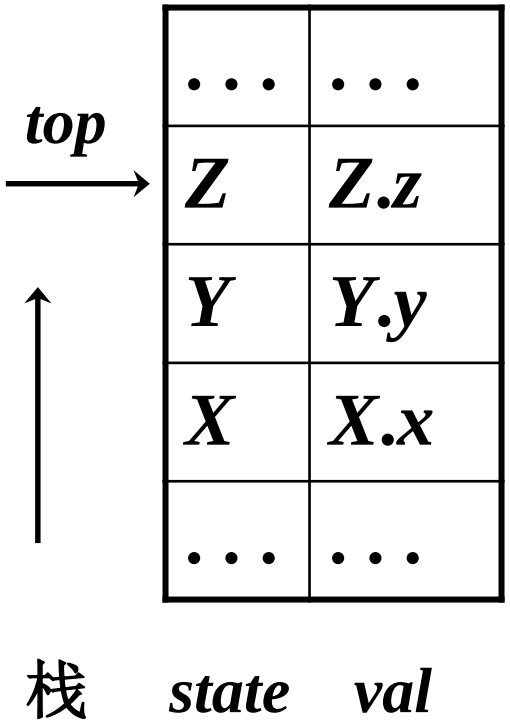


产 生 式	代 码 段
$L \rightarrow E$ n	*print* (*val*[*top*–1]);
$E \rightarrow E_1 + T$	*val*[*top*–2] = *val* [*top*–2] +*val*[*top*];
$E \rightarrow T$	
$T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow$ digit	$F.val =$ digit.*lexval*

注：栈顶位置指示器top的修改由原来的解析程序在语义动作执行后去做

自下而上的翻译

例　简单计算器的语法制导定义改成栈操作代码

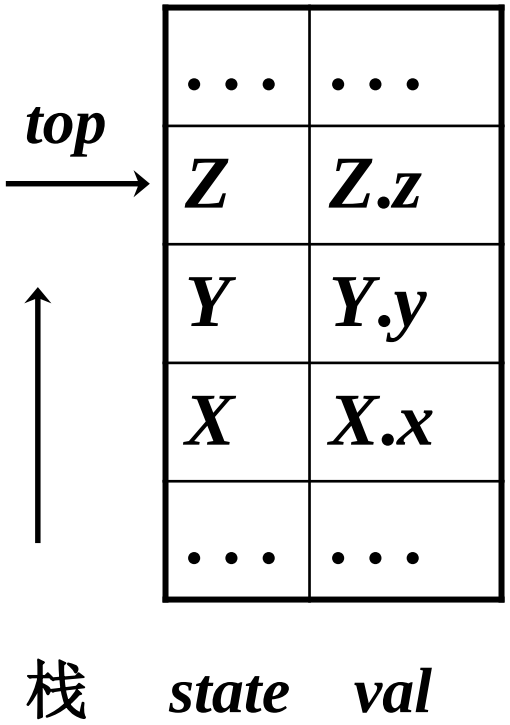


产 生 式	代 码 段
$L \rightarrow E$ n	*print* (*val*[*top*–1]);
$E \rightarrow E_1 + T$	*val*[*top*–2] = *val* [*top*–2] +*val*[*top*];
$E \rightarrow T$	
$T \rightarrow T_1 * F$	*val*[*top*–2] = *val* [*top*–2] × *val*[*top*];
$T \rightarrow F$	*T.val* = *F.val*
$F \rightarrow (E)$	*F.val* = *E.val*
$F \rightarrow$ digit	*F.val* = digit.*lexval*

注：栈顶位置指示器top的修改由原来的解析程序在语义动作执行后去做

自下而上的翻译

例　简单计算器的语法制导定义改成栈操作代码

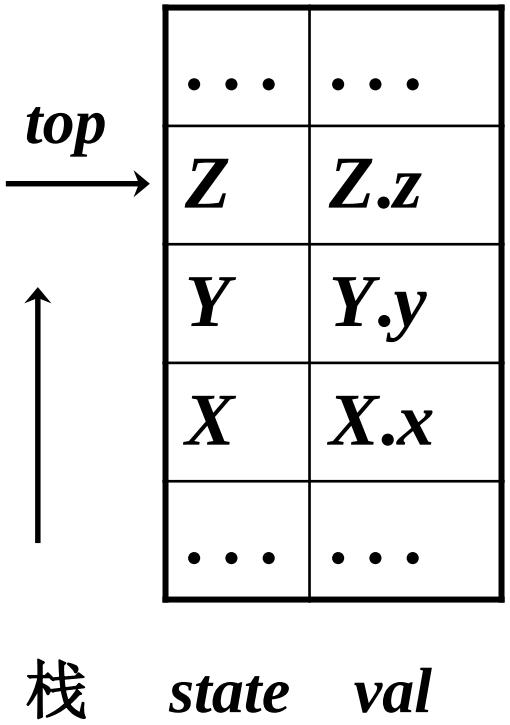


产 生 式	代 码 段
$L \rightarrow E$ n	*print* (*val*[*top*–1]);
$E \rightarrow E_1 + T$	*val*[*top*–2] = *val* [*top*–2] +*val*[*top*];
$E \rightarrow T$	
$T \rightarrow T_1 * F$	*val*[*top*–2] = *val* [*top*–2] × *val*[*top*];
$T \rightarrow F$	
$F \rightarrow (E)$	*F.val* = *E.val*
$F \rightarrow$ digit	*F.val* = digit.*lexval*

注：栈顶位置指示器top的修改由原来的解析程序在语义动作执行后去做

自下而上的翻译

例　简单计算器的语法制导定义改成栈操作代码

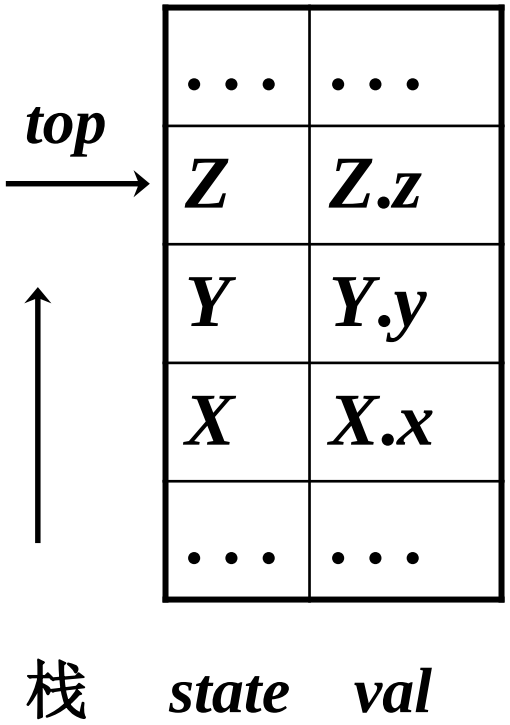


产 生 式	代 码 段
$L \rightarrow E$ n	*print* (*val*[*top*–1]);
$E \rightarrow E_1 + T$	*val*[*top*–2] = *val* [*top*–2] +*val*[*top*];
$E \rightarrow T$	
$T \rightarrow T_1 * F$	*val*[*top*–2] = *val* [*top*–2] × *val*[*top*];
$T \rightarrow F$	
$F \rightarrow (E)$	*val*[*top*–2] = *val* [*top*–1] ;
$F \rightarrow$ digit	*F.val* = digit.*lexval*

注：栈顶位置指示器top的修改由原来的解析程序在语义动作执行后去做

自下而上的翻译

例　简单计算器的语法制导定义改成栈操作代码



产 生 式	代 码 段
$L \rightarrow E$ n	*print* (*val*[*top*–1]);
$E \rightarrow E_1 + T$	*val*[*top*–2] = *val* [*top*–2] +*val*[*top*];
$E \rightarrow T$	
$T \rightarrow T_1 * F$	*val*[*top*–2] = *val* [*top*–2] × *val*[*top*];
$T \rightarrow F$	
$F \rightarrow (E)$	*val*[*top*–2] = *val* [*top*–1] ;
$F \rightarrow$ digit	

注：栈顶位置指示器top的修改由原来的解析程序在语义动作执行后去做

Bison举例bison-examples: config/expr.y


```
%{
#include <stdio.h>
#include <math.h>
%}

%union {
  float val;
}
%token NUMBER
%token PLUS MINUS MULT DIV EXPON
…
%left  MINUS PLUS
%left  MULT DIV
%right EXPON

%type  <val> exp NUMBER
%%
```

各种语义值类型和域置于共用体中

声明exp、Number的语义值是在val域

```
input   :       | input line
            ;
…
exp     : NUMBER          { $$ = $1;        }
        | exp PLUS  exp     { $$ = $1 + $3;   }
        | exp MINUS exp     { $$ = $1 - $3;   }
        | exp MULT  exp     { $$ = $1 * $3;   }
        | exp DIV   exp     { $$ = $1 / $3;   }
        | MINUS  exp %prec MINUS { $$ = -$2; }
        | exp EXPON exp     { $$ = pow($1,$3);}
        | LB exp RB         { $$ = $2;        }
        ;
%%
yyerror(char *message)
{  printf("%s\n",message);}
int main(int argc, char *argv[])
{  yyparse();  return(0);}
```

例题 8-6

为下面文法写一个语法制导的定义，用S的综合属性*val* 给出下面文法中S产生的二进制数的值。

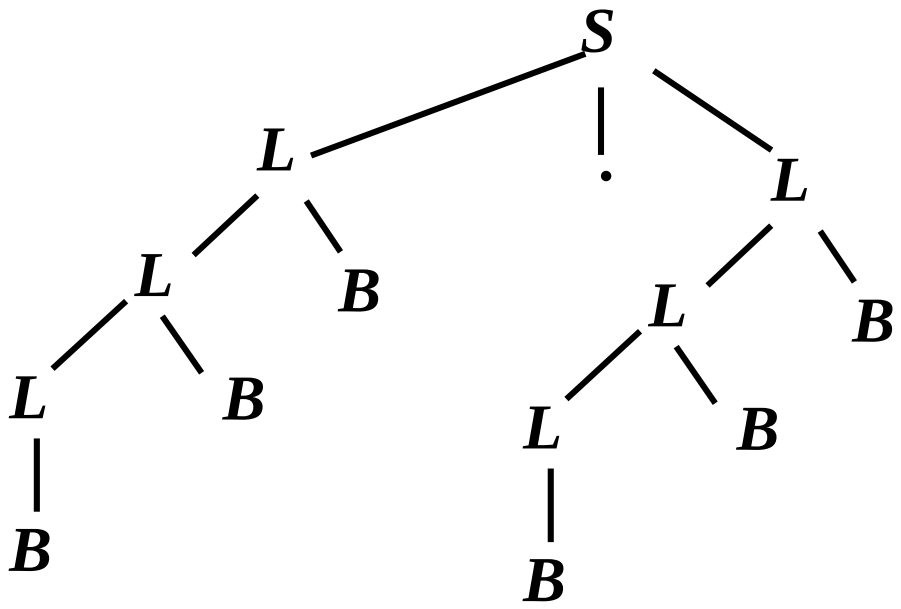
例如，输入101.101时，S.*val* = 5.625（可以修改文法）

若按$2^2+ 0 + 2^0 + 2^{-1} + 0 + 2^{-3}$来计算，该文法对小数点左边部分的计算不利，因为需要继承属性来确定每个B离开小数点的距离

$S \rightarrow L . L \mid L$

$L \rightarrow L\ B \mid B$

$B \rightarrow 0 \mid 1$



例题 8-6

为下面文法写一个语法制导的定义，用S的综合属性*val* 给出下面文法中S产生的二进制数的值。

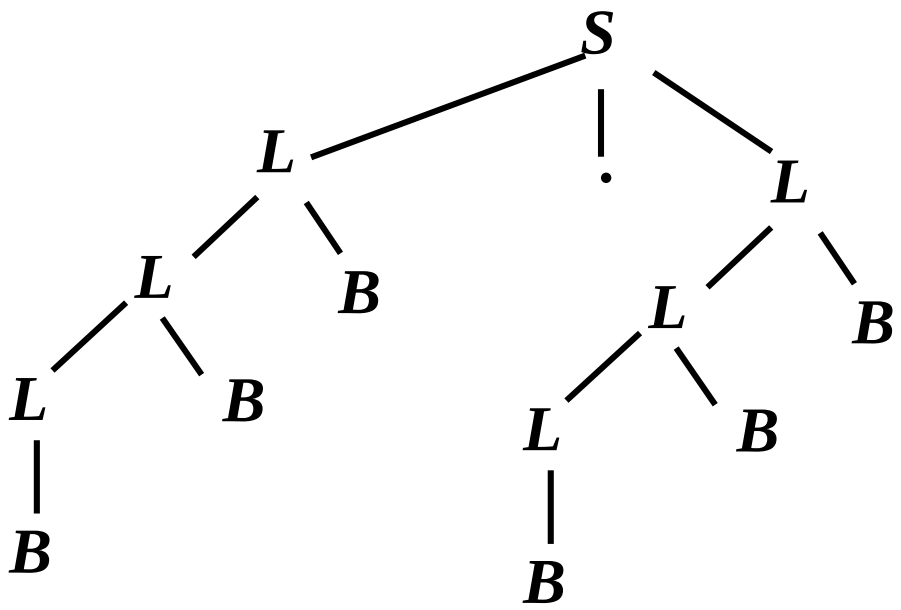
例如，输入101.101时，S.*val* = 5.625（可以修改文法）

若小数点左边按$(1 \times 2 + 0) \times 2 + 1$计算。该办法不能直接用于小数点右边，需改成$((1 \times 2 + 0) \times 2 + 1)/2^3$，这时需要综合属性来统计B的个数

$S \rightarrow L \,.\, L \mid L$

$L \rightarrow L\ B \mid B$

$B \rightarrow 0 \mid 1$



例题 8-6

为下面文法写一个语法制导的定义，用S的综合属性*val* 给出下面文法中S产生的二进制数的值。

例如，输入101.101时，S.*val* = 5.625（可以修改文法）

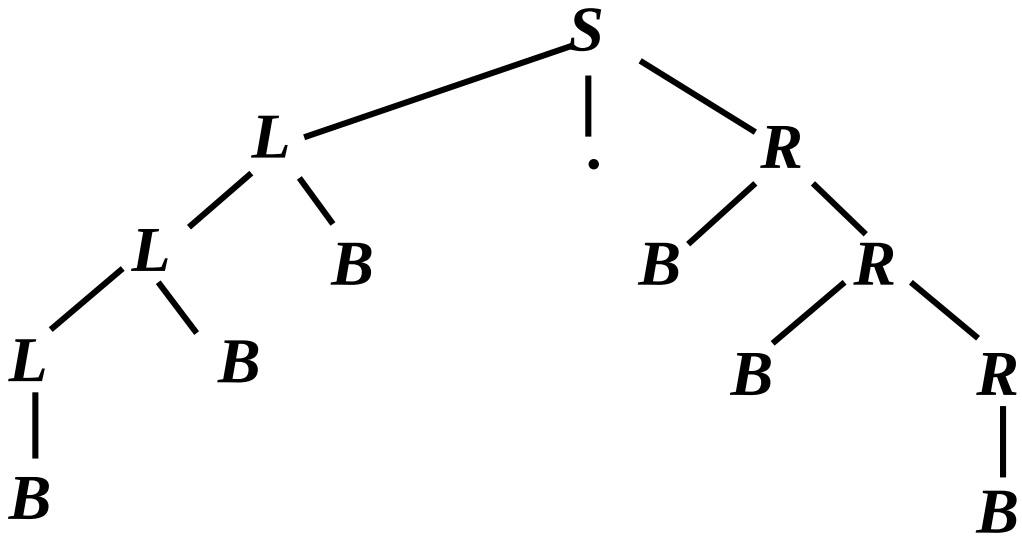
更清楚的办法是将文法改成下面的形式

$S \rightarrow L\ .\ R \mid L$

$L \rightarrow L\ B \mid B$

$R \rightarrow B\ R \mid B$

$B \rightarrow 0 \mid 1$



例题 8-6

为下面文法写一个语法制导的定义，用S的综合属性*val* 给出下面文法中S产生的二进制数的值。

例如，输入101.101时，S.*val* = 5.625（可以修改文法）

$S \rightarrow L\ .\ R$	S. *val* = L. *val* + R. *val*
$S \rightarrow L$	S. *val* = L. *val*
$L \rightarrow L_1\ B$	L. *val* = L_1. *val* ×2 + B. *val*
$L \rightarrow B$	L. *val* = B. *val*
$R \rightarrow B\ R_1$	R. *val* = R_1. *val* / 2 + B. *val* / 2
$R \rightarrow B$	R. *val* = B. *val* / 2
$B \rightarrow 0$	B. *val* = 0
$B \rightarrow 1$	B. *val* = 1

例题 8-7

给出把中缀表达式翻译成没有冗余括号的中缀表达式的语法制导定义。例如，因为+和*是左结合，

$((a*(b+c))*(d))$可以重写成$a*(b+c)*d$

两种方法：

- 先把括号都去掉，然后在必要的地方再加括号
- 去掉表达式中的冗余括号，保留必要的括号

例题 8-7

- 先把括号都去掉，然后在必要的地方再加括号

$S' \rightarrow E$ print ($E.code$)

$E \rightarrow E_1 + T$

if $T.op == plus$ then

$E.code = E_1.code \| \text{“+”} \| \text{“(”} \| T.code \| \text{“)”}$

else

$E.code = E_1.code \| \text{“+”} \| T.code;$

$E.op = plus$

$E \rightarrow T$ $E.code = T.code;$ $E.op = T.op$

例题 8-7

- 先把括号都去掉，然后在必要的地方再加括号

$T \rightarrow T_1 * F$

if (F. *op* == *plus*) or (F. *op* == *times*) then
    if T_1. *op* == *plus* then
        T. *code* = "(" || T_1. *code* || ")" || "*" || "(" || F. *code* || ")"
    else
        T. *code* = T_1. *code* || "*" || "(" || F. *code* || ")"
else if T_1. *op* = *plus* then
    T. *code* = "(" || T_1. *code* || ")" || "*" || F. *code*
else
    T. *code* = T_1. *code* || "*" || F. *code*;
T. *op* = *times*

例题 8-7

- 先把括号都去掉，然后在必要的地方再加括号

$T \rightarrow F$

$T.code = F.code;\ T.op = F.op$

$F \rightarrow id$

$F.code = id.lexeme;\ F.op = id$

$F \rightarrow (\ E\ )$

$F.code = E.code;\ F.op = E.op$

例题 8-7

- 去掉表达式中的冗余括号，保留必要的括号
  - 给*E*，*T*和*F*两个继承属性*left_op*和*right_op*分别表示左右两侧算符的优先级
  - 给它们一个综合属性*self_op*表示自身主算符的优先级
  - 再给一个综合属性*code*表示没有冗余括号的代码
  - 分别用1和2表示加和乘的优先级，用3表示id和(*E*)的优先级，用0表示左侧或右侧没有运算对象的情况

例题 8-7

$S' \rightarrow E$	$E.\ left_op = 0;\ E.\ right_op = 0;\ print\ (\ E.\ code\ )$
$E \rightarrow E_1 + T$	$E_1.\ left_op = E.\ left_op;\ E_1.\ right_op = 1;$
	$T.\ left_op = 1;\ T.\ right_op = E.\ right_op;$
	$E.code = E_1.code \| \| \text{“+”} \| \| T.\ code\ ;\ E.\ self_op = 1;$
$E \rightarrow T$	$T.\ left_op = E.\ left_op;$
	$T.\ right_op = E.\ right_op;$
	$E.\ code = T.\ code;\ \ E.\ self_op = T.\ self_op$
$T \rightarrow T_1 * F$	. . .
$T \rightarrow F$	. . .
$F \rightarrow \text{id}$	$F.\ code = id.\ lexeme;\ \ F.\ self_op = 3$

例题 8-7

$F \rightarrow (E)$

$E.left_op = 0; E.right_op = 0;$

$F.self_op =$ if $(F.left_op < E.self_op)$ and $(E.self_op >= F.right_op)$ then $E.self_op$ else 3

$F.code =$ if $(F.left_op < E.self_op)$ and $(E.self_op >= F.right_op)$ then $E.code$ else "(" || $E.code$ || ")"



中国科学技术大学
University of Science and Technology of China

L属性定义的自下而上计算

- 删除翻译方案中的内嵌动作
- 继承属性的计算



*L*属性的自下而上计算

在自下而上解析的框架中实现*L*属性定义的方法

- 它能实现任何基于LL(1)文法的*L*属性定义
- 也能实现许多（但不是所有的）基于LR(1) 的*L*属性定义

删除翻译方案中的内嵌动作

中缀表达式翻译成后缀表达式

$E \rightarrow T\ R$

$R \rightarrow +\ T\ \{print\ (\text{'+'})\}R_1 \mid -\ T\ \{print\ (\text{'-'})\}R_1 \mid \varepsilon$

$T \rightarrow \text{num}\ \{print(\text{num}.val)\}$

- 在文法中加入推出ε的标记非终结符，让每个内嵌动作由不同的标记非终结符M代表，并把该动作移到产生式$M \rightarrow \varepsilon$的右端 （继承属性=>综合属性）

$E \rightarrow T\ R$

$R \rightarrow +\ T\ M\ R_1 \mid -\ T\ N\ R_1 \mid \varepsilon$

$T \rightarrow \text{num}\ \{print\ (\text{num}.val)\}$

$M \rightarrow \varepsilon\ \{print\ (\text{'+'})\}$

$N \rightarrow \varepsilon\ \{print\ (\text{'-'})\}$

YACC会按这种方法来处理输入的文法，即为内嵌的语义动作引入ε产生式

L属性的自下而上计算

bison-examples: config/exprL.y

```
input   : …
   | input{ lineno ++; printf("Line %d:\t", lineno);}  line { printf("*"); } ;
```

- $$表示产生式LHS(left-hand side)符号的语义值，$1, $2…依次为RHS(right-hand side)中符号的语义值，本例中line的语义值通过$3 来引用
- Bison自动为上述2个蓝色的嵌入动作引入2个产生ε的标记非终结符，对应case 3和4

src/exprL.tab.c

```
 case 4:
/* Line 1806 of yacc.c  */
#line 36 "config/exprL.y"
    { printf("*"); }
    break;
```

```
yyreduce:
  /* yyn is the number of a rule to reduce with.  */
 …
 YY_REDUCE_PRINT (yyn);
 switch (yyn) { …
    case 3:
    /* Line 1806 of yacc.c  */
    #line 32 "config/exprL.y"
            { lineno ++; printf("Line %d:\t", lineno);
              }
            break;
```

产生式编号

L属性定义的自下而上计算

- 删除翻译方案中的内嵌动作
- 继承属性的计算

继承属性在解析栈中

情况1 属性位置可预测

例8-8 int p, q, r

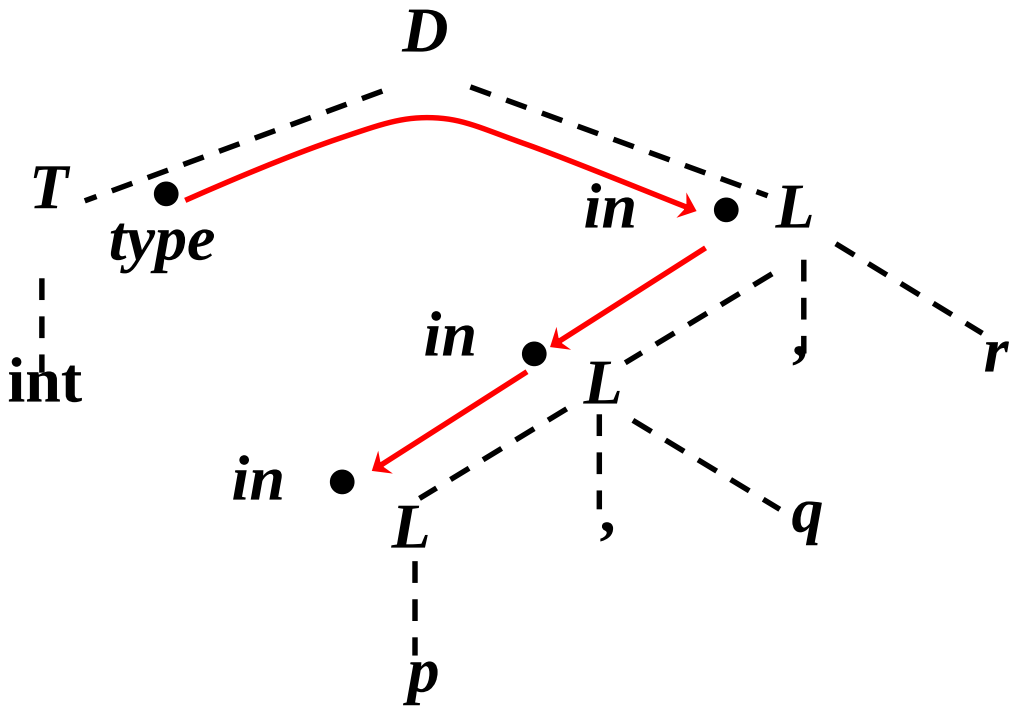
$D \rightarrow T$ {*L.in* = *T.type*}
　　L

$T \rightarrow$ int {*T. type* = *integer*}

$T \rightarrow$ real {*T. type* = *real*}

$L \rightarrow$ {L_1.*in* = *L.in* }
　L_1, id {*addtype* (id.*entry*, *L.in*)}

$L \rightarrow$ id {*addtype* (id.*entry*, *L.in*)}



继承属性值
已在解析栈中

继承属性在分析栈中

情况1 属性位置可预测

例8-8 int p, q, r

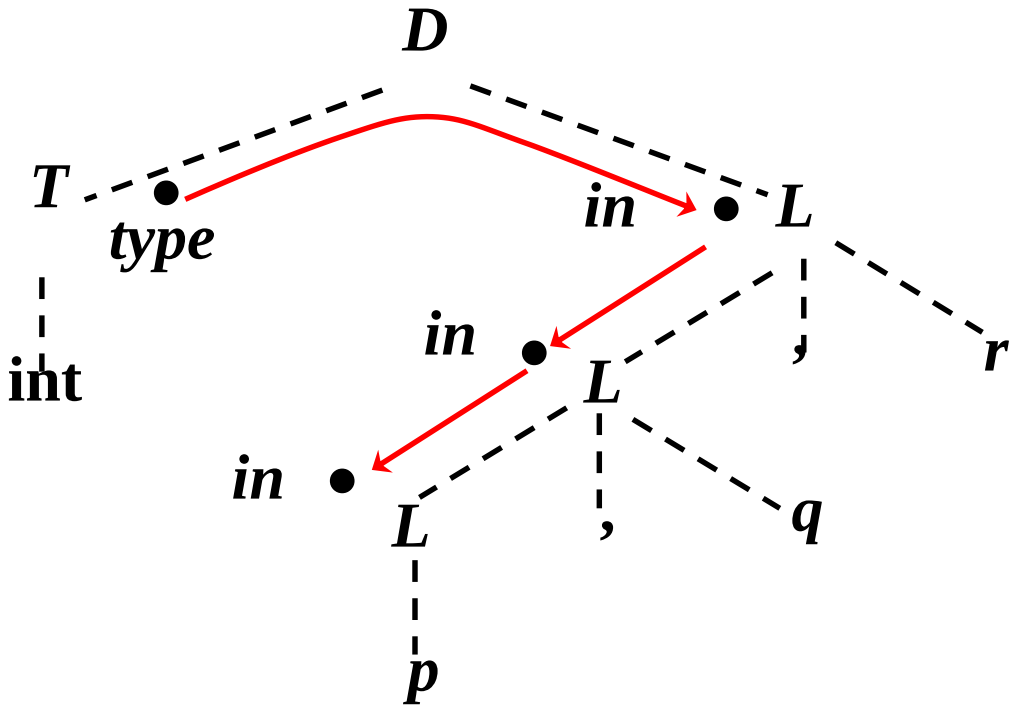
$D \rightarrow T$ {~~*L.in = T.type*~~}
　　L

$T \rightarrow$ int {*T. type = integer*}

$T \rightarrow$ real {*T. type = real*}

$L \rightarrow$ {~~L_1*.in = L.in*~~}
　L_1, id {*addtype* (id.*entry*, *L.in*)}

$L \rightarrow$ id {*addtype* (id.*entry*, *L.in*)}

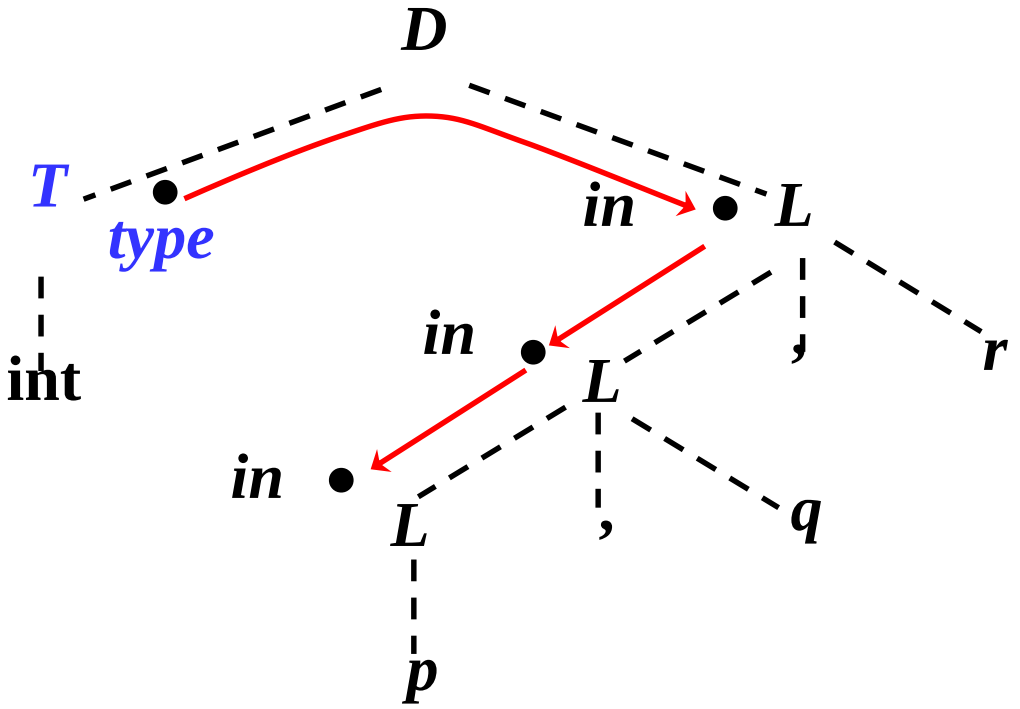


略去继承属性的计算
引用继承属性的地方改成
引用其他符号的综合属性

继承属性在分析栈中

情况1 属性位置可预测

产 生 式	代 码 段
$D \rightarrow TL$	
$T \rightarrow$ int	$val[top] = integer$
$T \rightarrow$ real	$val[top] = real$
$L \rightarrow L_1$, id	$addType(val[top], val[top-3])$;
$L \rightarrow$ id	$addType(val[top], val[top-1])$;



略去继承属性的计算
引用继承属性的地方改成
引用其他符号的综合属性

YACC中的继承属性定义

在内嵌动作代码中设置该文法符号的语义值

bison-examples: config/exprL1.y

```
line   : …
       | NUMBER { …
                  $<val>lineno = $1;   // val是%union中声明的语义值类型
                  // $<val>$ = $1;        // 该语义动作代码未指定名字时
                  …
                  } [lineno]
       exp EOL { …
                  printf("Line %d: %g\n", (int) $<val>lineno, $3);
                  …
       }
```

指明lineno的语义值所在的共用体的域

给内嵌语义动作对应的标记非终结符命名

YACC中的继承属性定义

在内嵌代码中使用存储在栈中任意固定相对位置的语义值

bison-examples: config/midrule.y

```
exp: a_1 a_2 { $<val>$ = 3; } { $<val>$ = $<val>3 + 1; } a_5
    sum_of_the_five_previous_values
   {
     USE (($1, $2, $<foo>3, $<foo>4, $5));
     printf ("%d\n", $6);
   }

sum_of_the_five_previous_values:
   {
     $$ = $<val>0 + $<val>-1 + $<val>-2 + $<val>-3 + $<val>-4;
   }
```

$<val>0、 $<val>-1、$<val>-2、$<val>-3、$<val>-4分别表示栈中a_5、{ $<val>$ = $<val>3 + 1; }、{ $<val>$ = 3; }、a_2、a_1文法符号的语义值

继承属性在分析栈中

情况2 属性位置不可预测

继承属性值
已在分析栈中

$S \rightarrow aAC$ $\quad C.i = A.s$

$S \rightarrow bABC$ $\quad C.i = A.s$

$C \rightarrow c$ $\quad C.s = g(C.i)$

*A*和*C*之间可能有B，也可能没有B，*C.i* 的值有2种可能

□ 增加标记非终结符，使得位置可以预测

$S \rightarrow aAC$ $\quad C.i = A.s$

$S \rightarrow bABMC$ $\quad M.i = A.s;\ C.i = M.s$

$C \rightarrow c$ $\quad C.s = g(C.i)$

$M \rightarrow \varepsilon$ $\quad M.s = M.i$

模拟继承属性的计算

- 继承属性是综合属性的函数

$S \rightarrow aAC$　　$C.i = f(A.s)$

$C \rightarrow c$　　$C.s = g(C.i)$

继承属性不直接等于某个综合属性

- 增加标记非终结符，把$f(A.s)$的计算移到对标记非终结符归约时进行

$S \rightarrow aANC$　　$N.i = A.s;\ C.i = N.s$

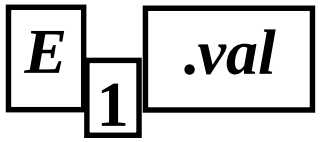
$N \rightarrow \varepsilon$　　$N.s = f(N.i)$

$C \rightarrow c$　　$C.s = g(C.i)$

L属性定义的自下而上计算

例8-9 数学排版语言EQN

E sub 1 .val



$S \rightarrow B$

$B \rightarrow B_1\ B_2$

$B \rightarrow B_1$ sub B_2

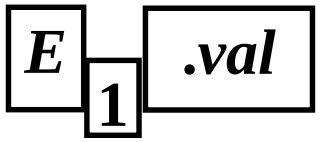
$B \rightarrow$ text

产 生 式	语 义 规 则
$S \rightarrow LB$	$B.ps = L.s$; $S.ht = B.ht$
$L \rightarrow \varepsilon$	$L.s = 10$ 将$B.ps$存入栈中，便于引用
$B \rightarrow B_1\ MB_2$	$B_1.ps = B.ps$; $M.i = B.ps$; $B_2.ps = M.s$;$B.ht = max(B_1.ht, B_2.ht)$
$M \rightarrow \varepsilon$	$M.s = M.i$
$B \rightarrow B_1$ sub NB_2	$B_1.ps = B.ps$; $N.i = B.ps$; $B_2.ps = N.s$; $B.ht = disp\ (B_1.ht, B_2.ht)$
$N \rightarrow \varepsilon$	$N.s = shrink(N.i)$
$B \rightarrow$ text	$B.ht =$ text.$h \times B.ps$

L属性定义的自下而上计算

例8-9 数学排版语言EQN

E sub 1 .val



$S \rightarrow B$

$B \rightarrow B_1\ B_2$

$B \rightarrow B_1$ sub B_2

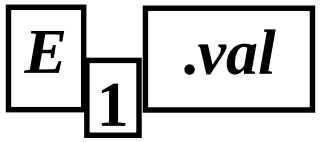
$B \rightarrow$ text

产 生 式	语 义 规 则
$S \rightarrow LB$	$B.ps = L.s$; $S.ht = B.ht$
$L \rightarrow \varepsilon$	$L.s = 10$ 将$B.ps$存入栈中，便于引用
$B \rightarrow B_1\ MB_2$	$B_1.ps = B.ps$; $M.i = B.ps$; $B_2.ps = M.s$; $B.ht = max(B_1.ht, B_2.ht)$
$M \rightarrow \varepsilon$	$M.s = M.i$ 单纯为了属性位置可预测
$B \rightarrow B_1$ sub NB_2	$B_1.ps = B.ps$; $N.i = B.ps$; $B_2.ps = N.s$; $B.ht = disp\ (B_1.ht, B_2.ht)$
$N \rightarrow \varepsilon$	$N.s = shrink(N.i)$
$B \rightarrow$ text	$B.ht =$ text.$h \times B.ps$

L属性定义的自下而上计算

例8-9 数学排版语言EQN

E sub 1 .val



$S \rightarrow B$

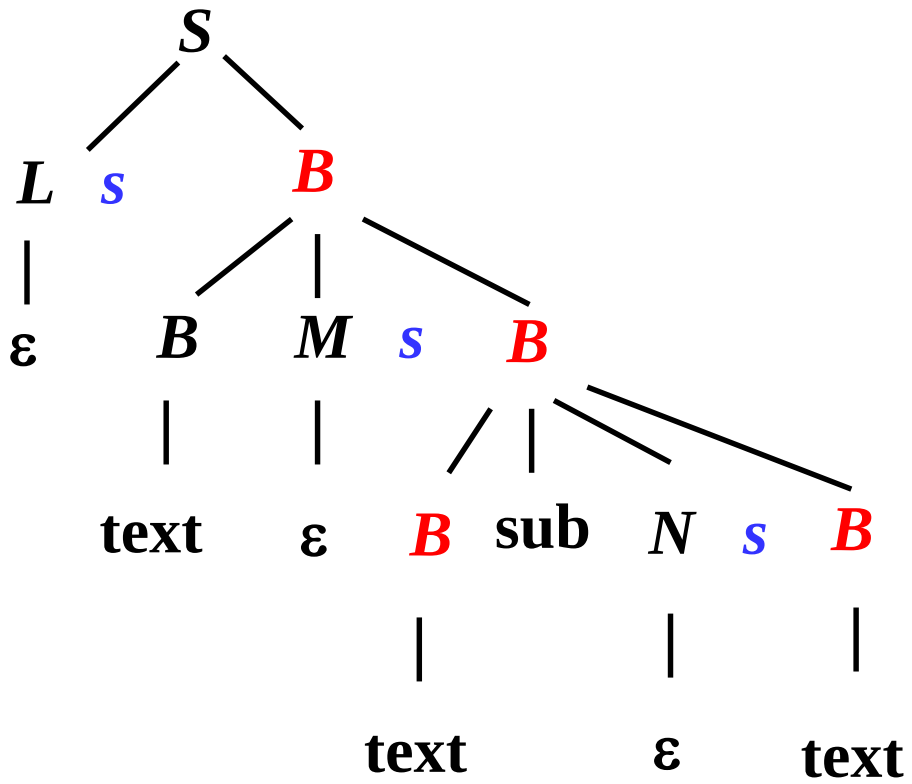
$B \rightarrow B_1\ B_2$

$B \rightarrow B_1$ sub B_2

$B \rightarrow$ text

产 生 式	语 义 规 则
$S \rightarrow LB$	$B.ps = L.s$; $S.ht = B.ht$
$L \rightarrow \varepsilon$	$L.s = 10$ 将$B.ps$存入栈中，便于引用
$B \rightarrow B_1\ MB_2$	$B_1.ps = B.ps$; $M.i = B.ps$; $B_2.ps = M.s$; $B.ht = max(B_1.ht, B_2.ht)$
$M \rightarrow \varepsilon$	$M.s = M.i$ 单纯为了属性位置可预测
$B \rightarrow B_1$ sub NB_2	$B_1.ps = B.ps$; $N.i = B.ps$; $B_2.ps = N.s$; $B.ht = disp\ (B_1.ht, B_2.ht)$
$N \rightarrow \varepsilon$	$N.s = shrink(N.i)$ 兼有计算功能
$B \rightarrow$ text	$B.ht =$ text.$h \times B.ps$

EQN：自下而上计算的实现

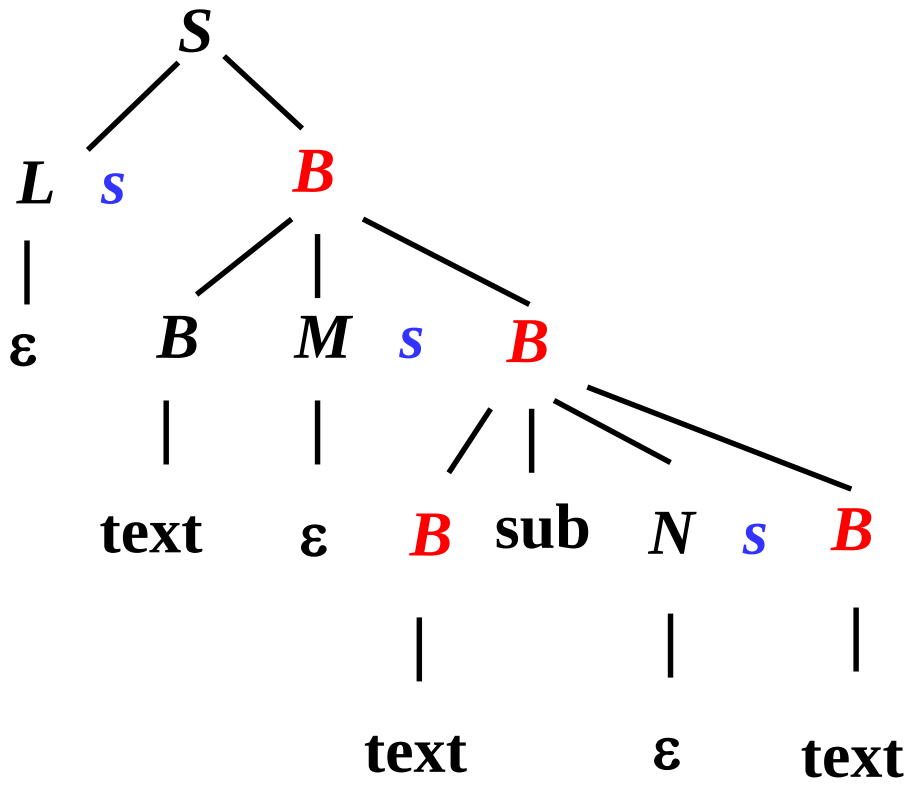


产 生 式	语 义 规 则
$S \rightarrow LB$	$B.ps = L.s$; $S.ht = B.ht$
$L \rightarrow \varepsilon$	$L.s = 10$
$B \rightarrow B_1\ MB_2$	$B_1.ps = B.ps$; $M.i = B.ps$; $B_2.ps = M.s$; $B.ht = max(B_1.ht, B_2.ht)$
$M \rightarrow \varepsilon$	$M.s = M.i$
$B \rightarrow B_1$ sub NB_2	$B_1.ps = B.ps$; $N.i = B.ps$; $B_2.ps = N.s$; $B.ht = disp\ (B_1.ht, B_2.ht)$
$N \rightarrow \varepsilon$	$N.s = shrink(N.i)$
$B \rightarrow$ text	$B.ht =$ text.$h \times B.ps$

在text归约成B时，B的ps属性都在次栈顶位置

继承属性的值为栈中某个综合属性的值时，栈中只保存综合属性的值

EQN：自下而上计算的实现

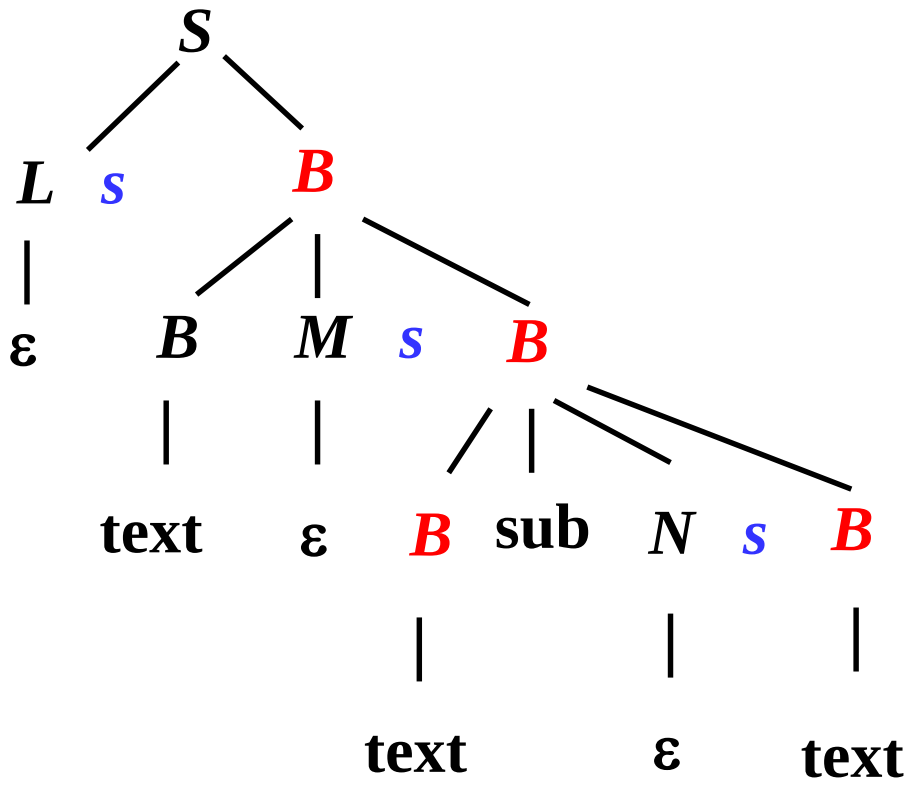


产 生 式	语 义 规 则
$S \rightarrow LB$	$val[top-1] = val[top]$
$L \rightarrow \varepsilon$	$L.s = 10$
$B \rightarrow B_1\ MB_2$	$B_1.ps = B.ps;\ \ M.i = B.ps;$ $B_2.ps = M.s;B.ht = max(B_1.ht, B_2.ht)$
$M \rightarrow \varepsilon$	$M.s = M.i$
$B \rightarrow B_1$ sub NB_2	$B_1.ps = B.ps;\ \ N.i = B.ps;$ $B_2.ps = N.s;\ B.ht = disp\ (B_1.ht, B_2.ht)$
$N \rightarrow \varepsilon$	$N.s = shrink(N.i)$
$B \rightarrow$ text	$B.ht = \text{text}.h \times B.ps$

在text归约成B时，B的ps属性都在次栈顶位置

$B.ps = L.s;\ \ S.ht = B.ht$

EQN：自下而上计算的实现

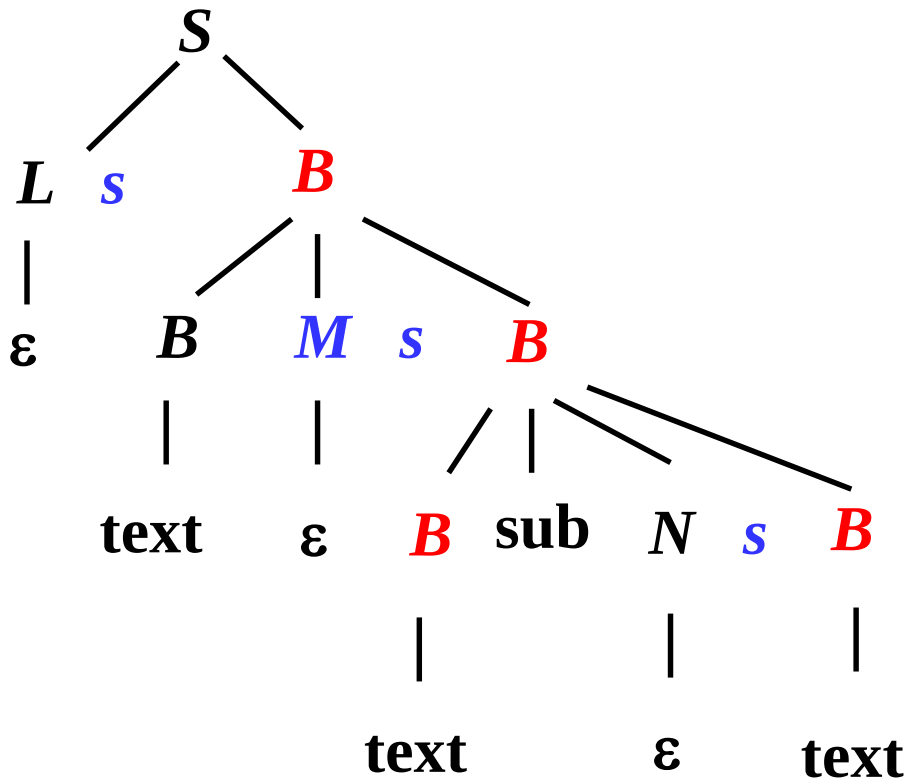


产 生 式	语 义 规 则
$S \rightarrow LB$	$val[top-1] = val[top]$
$L \rightarrow \varepsilon$	$val[top+1] = 10$
$B \rightarrow B_1\ MB_2$	$B_1.ps = B.ps;\ \ M.i = B.ps;$ $B_2.ps = M.s;B.ht = max(B_1.ht, B_2.ht)$
$M \rightarrow \varepsilon$	$M.s = M.i$
$B \rightarrow B_1$ sub NB_2	$B_1.ps = B.ps;\ \ N.i = B.ps;$ $B_2.ps = N.s;\ B.ht = disp\ (B_1.ht, B_2.ht)$
$N \rightarrow \varepsilon$	$N.s = shrink(N.i)$
$B \rightarrow$ text	$B.ht = \text{text}.h \times B.ps$

在text归约成B时，B的ps属性都在次栈顶位置

$L.s = 10$

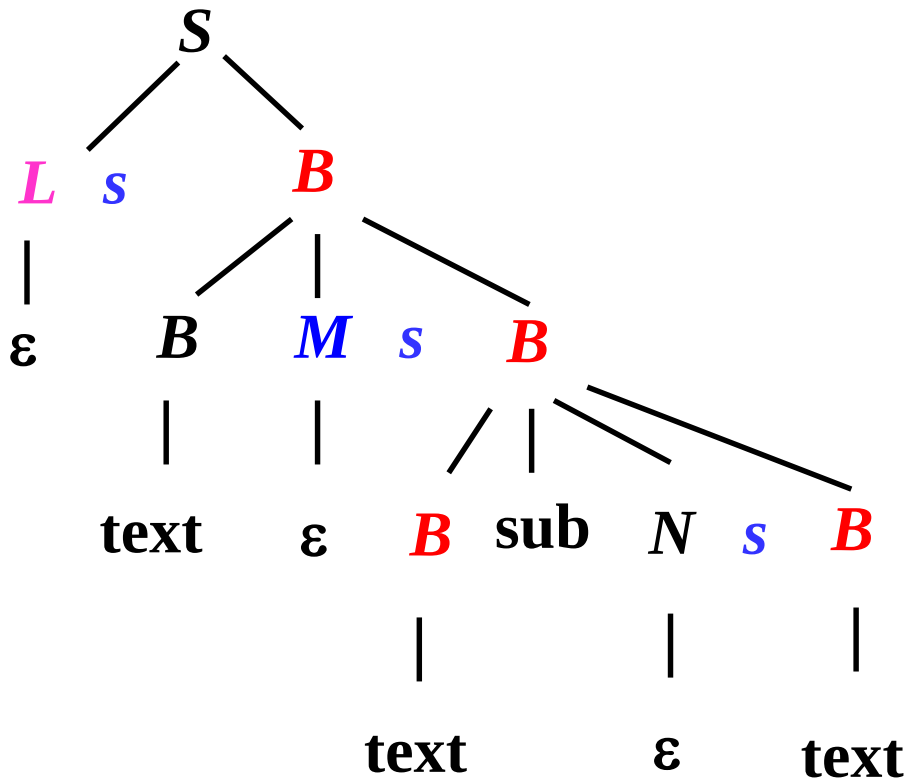
EQN：自下而上计算的实现



产 生 式	语 义 规 则
$S \rightarrow LB$	$val[top-1] = val[top]$
$L \rightarrow \varepsilon$	$val[top+1] = 10$
$B \rightarrow B_1\ MB_2$	$val[top-2] = max(val[top-2], val[top])$
$M \rightarrow \varepsilon$	$M.s = M.i$
$B \rightarrow B_1$ sub NB_2	$B_1.ps = B.ps;\ \ N.i = B.ps;$ $B_2.ps = N.s;\ B.ht = disp\ (B_1.ht, B_2.ht)$
$N \rightarrow \varepsilon$	$N.s = shrink(N.i)$
$B \rightarrow$ text	$B.ht = \text{text}.h \times B.ps$

$B_1.ps = B.ps;\ M.i = B.ps; B_2.ps = M.s; B.ht = max(B_1.ht, B_2.ht)$

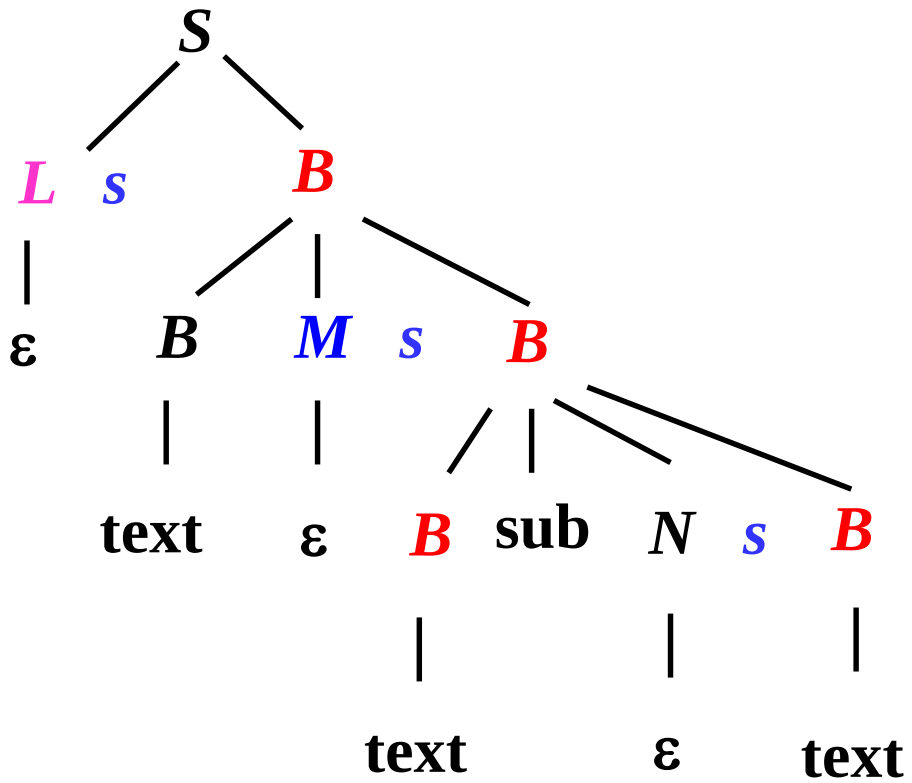
EQN：自下而上计算的实现



产 生 式	语 义 规 则
$S \rightarrow LB$	$val[top-1] = val[top]$
$L \rightarrow \varepsilon$	$val[top+1] = 10$
$B \rightarrow B_1\ MB_2$	$val[top-2] = max(val[top-2], val[top])$
$M \rightarrow \varepsilon$	$val[top+1] = val[top-1]$
$B \rightarrow B_1$ sub NB_2	$B_1.ps = B.ps;\ N.i = B.ps;$ $B_2.ps = N.s;\ B.ht = disp\ (B_1.ht, B_2.ht)$
$N \rightarrow \varepsilon$	$N.s = shrink(N.i)$
$B \rightarrow$ text	$B.ht = \text{text}.h \times B.ps$

$M.i = B.ps;\ M.s = M.i$

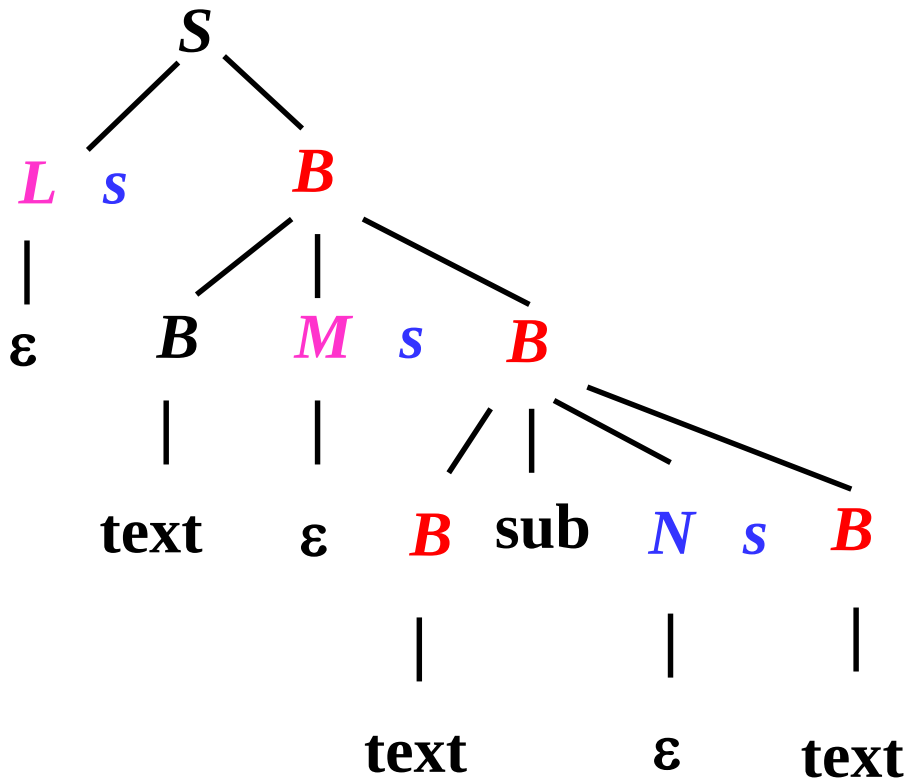
EQN：自下而上计算的实现



产 生 式	语 义 规 则
$S \rightarrow LB$	$val[top-1] = val[top]$
$L \rightarrow \varepsilon$	$val[top+1] = 10$
$B \rightarrow B_1\ MB_2$	$val[top-2] = max(val[top-2], val[top])$
$M \rightarrow \varepsilon$	$val[top+1] = val[top-1]$
$B \rightarrow B_1$ sub NB_2	$val[top-3] = disp\ (val[top-3], val[top])$
$N \rightarrow \varepsilon$	$N.s = shrink(N.i)$
$B \rightarrow$ text	$B.ht =$ text$.h \times B.ps$

$B_1.ps = B.ps;\ \ N.i = B.ps; B_2.ps = N.s;\ B.ht = disp\ (B_1.ht, B_2.ht)$

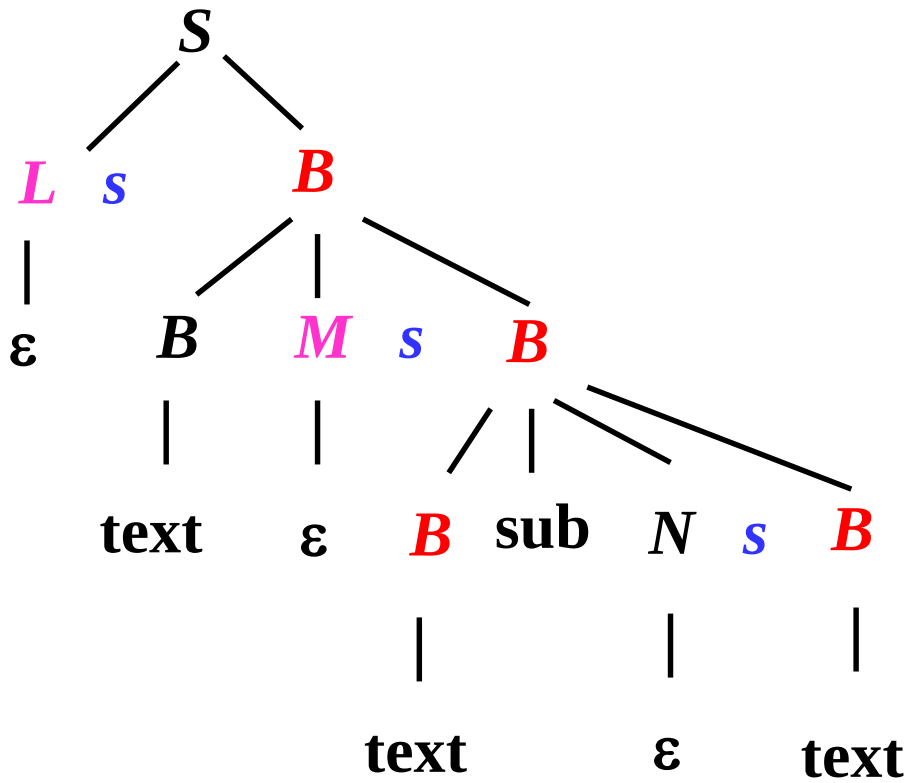
EQN：自下而上计算的实现



产 生 式	语 义 规 则
$S \rightarrow LB$	$val[top-1] = val[top]$
$L \rightarrow \varepsilon$	$val[top+1] = 10$
$B \rightarrow B_1\ MB_2$	$val[top-2] = max(val[top-2], val[top])$
$M \rightarrow \varepsilon$	$val[top+1] = val[top-1]$
$B \rightarrow B_1$ sub NB_2	$val[top-3] = disp\ (val[top-3], val[top])$
$N \rightarrow \varepsilon$	$val[top+1] = shrink(val[top-2])$
$B \rightarrow$ text	$B.ht = \text{text}.h \times B.ps$

$N.i = B.ps;\ \ N.s = shrink(N.i)$

EQN：自下而上计算的实现



产 生 式	语 义 规 则
$S \rightarrow LB$	$val[top-1] = val[top]$
$L \rightarrow \varepsilon$	$val[top+1] = 10$
$B \rightarrow B_1\ MB_2$	$val[top-2] = max(val[top-2], val[top])$
$M \rightarrow \varepsilon$	$val[top+1] = val[top-1]$
$B \rightarrow B_1$ sub NB_2	$val[top-3] = disp\ (val[top-3], val[top])$
$N \rightarrow \varepsilon$	$val[top+1] = shrink(val[top-2])$
$B \rightarrow$ text	$val[top] = val[top] \times val[top-1]$

$B.ht = \text{text}.h \times B.ps$